5. On-disk-storage Data Management

Operating Systems (OS)
Master in Innovation and Research in Informatics (MIRI)
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)
2025/2026 Q1

FIB

Contents

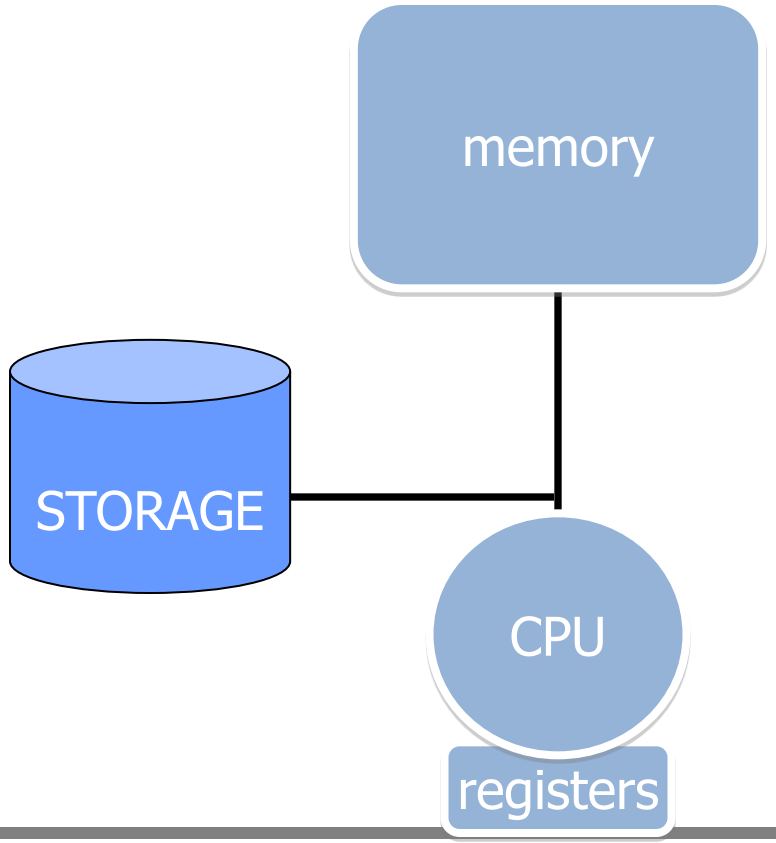
- Introduction
- Desktop Computer Storage
  - Storage devices
  - File Systems
- WSC Storage:
  - Distributed File Systems
  - Distributed Databases
  - Examples

Introduction

- What is Data Management?
  - Data is any information useful for users.
  - Operating system must manage this data.
  - Where do we save this data?
  - How do we access this data?

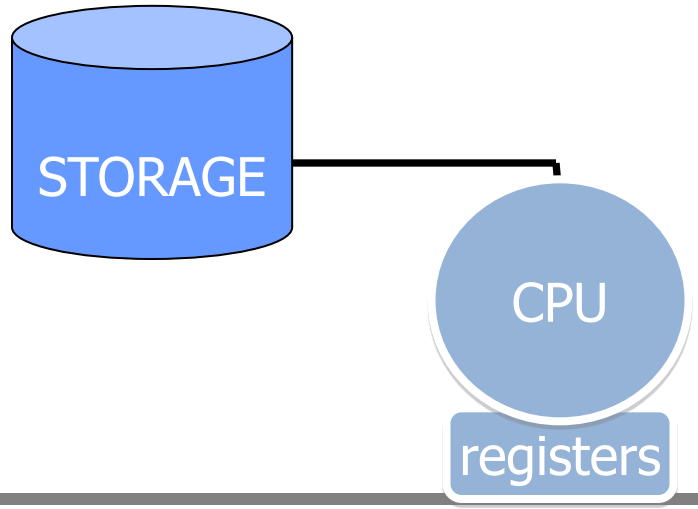
Introduction

- How do we manage data in a Desktop Computer?
  - Two options:
    - Memory
    - Secondary storage



Introduction

- How do we manage data in a Desktop Computer?
  - Two options:
    - Memory
    - Secondary storage ← **Focus on this**



Contents

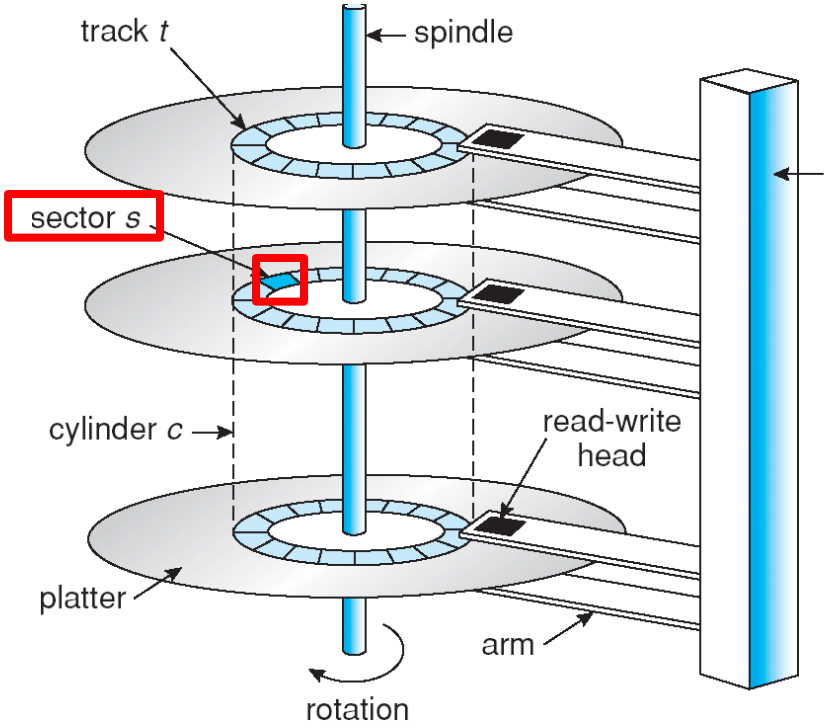
- Introduction
- **Desktop Computer Storage**
  - Storage devices
  - File Systems
- WSC Storage:
  - Distributed File Systems
  - Distributed Databases
  - Examples

Desktop Computer Storage

- Storage device → physical hardware device
  - store data in a permanent way
- Granularity:
  - blocks of data (instead of bytes used by memory seen previously)
- HDD
- SSD

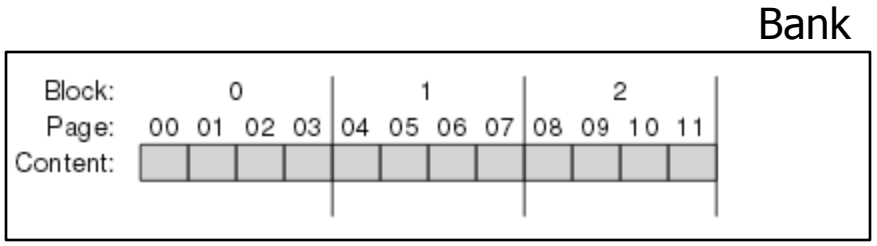
Hard Disk Drive

- Typical Hard Disk Drive
- Granularity: Sector → block of data
  - Sector: 512 - 4096b
- Operations
  - Read a sector
  - Write a sector
- Access mode: CHS
- LBA- Logical Block Address
- Operations
  - Read a block
  - Write a block



Solid-state Storage Device (SSD)

- No mechanical or moving parts like HDD
- Transistors based
  - But no information lost on power loss
- Organization
  - Flash chips organized into **Banks/Planes**
  - Each bank has **blocks**
    - **128 – 256 KB**
  - Each block has **pages**
    - **2 – 4 KB**

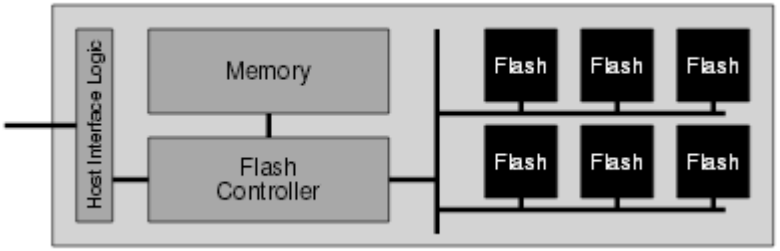


SSD: Operations and Architecture

- Operations:
  - Read (a page)
  - Program (a page)
    - Write content to a page (change some 1´s → 0´s)
    - Same page can NOT be written again → erase needed
  - Erase (a block)
    - Destroy contents of the block (set values to 1)



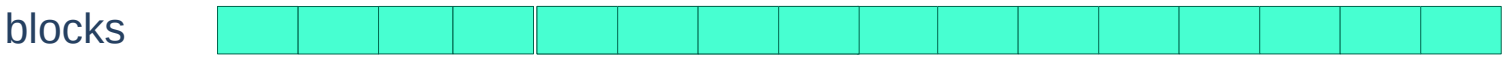
- Architecture
  - Must look like a HDD
  - Read/write "blocks"
    - Sectors
    - → Flash Translation Layer (FTL)



How does the OS access these devices?

- Acccess to device: I/O bus
  - Specific hardware operations (in/out)

- View of the device as a sequence of **blocks**
  - Mapping of disk´s sectors → OS´s block

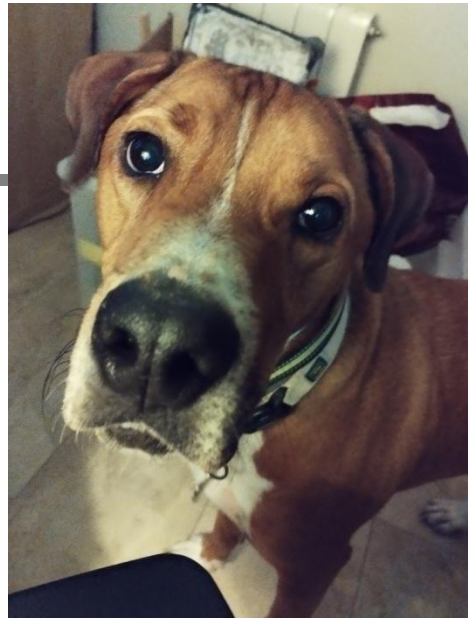


  - Independence of the device

Contents

- Introduction
- Desktop Computer Storage
  - Storage devices
  - **File Systems**
- WSC Storage:
  - Distributed File Systems
  - Distributed Databases

File System



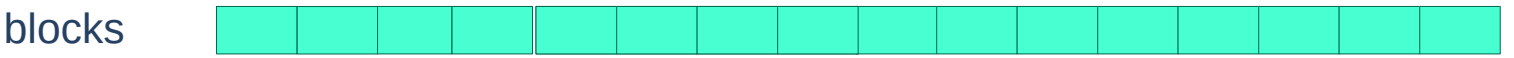
- Thanks to these "OS's blocks"
  - OS is independent from the device
  - OS sees an array of blocks
  - OS offers this array to the user
- But, what happens?

  → **Users dont care about the blocks**
- She wants to save her own data (a picture)
  - A bunch of bytes

→ Therefore, OS offers her an abstraction
  - To store her bytes, her data → Files.
    - OS stores the file in a persistent way

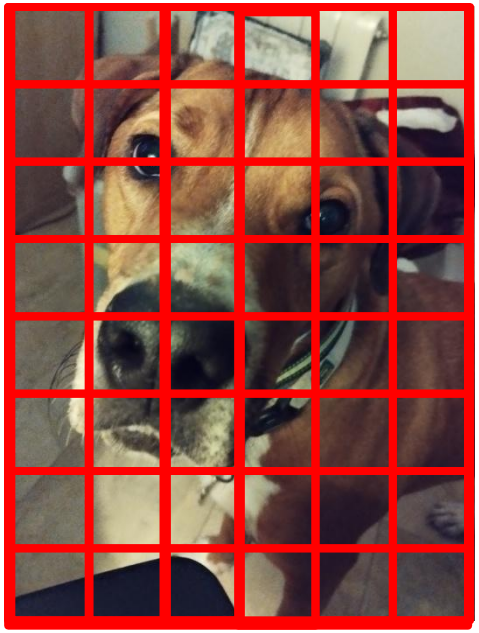
File System

- How does OS organize data/files in a device?
  - Through a FileSystem
  - On-disk data structures!

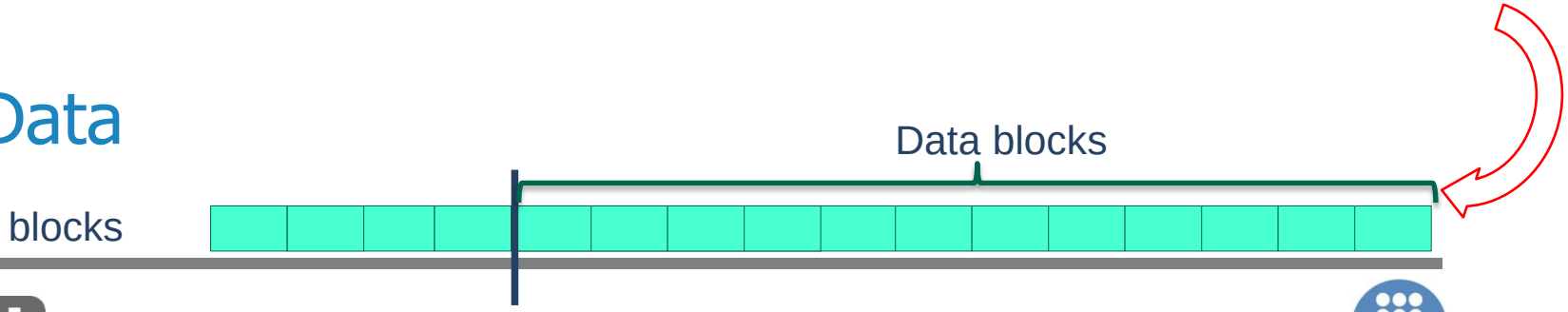


File System

- How does OS organize data/files in a device?
  - Through a FileSystem
  - On-disk data structures!

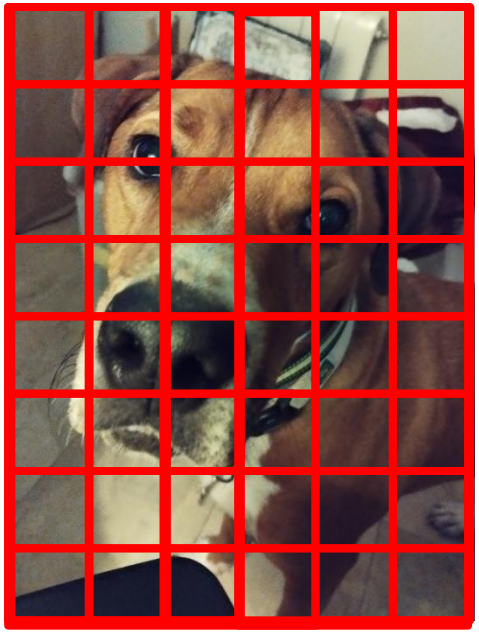


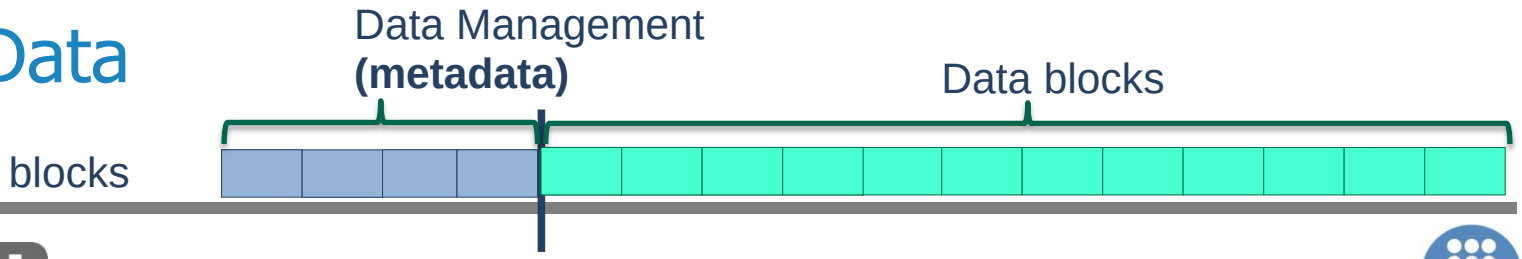
- Data



File System

- How does OS organize data/files in a device?
  - Through a FileSystem
  - On-disk data structures!
- Block managament (Metadata)
  - Knowledge about free/used blocks
  - Which blocks are used by a file?
    - Continguous
    - Linked list
    - Indexed (inodes)
- Data





File System properties

- File System needs to solve 2 problems:
  - **Naming**: How does the user locates her data?
  - **Operations**: How does the user access her data?

Naming

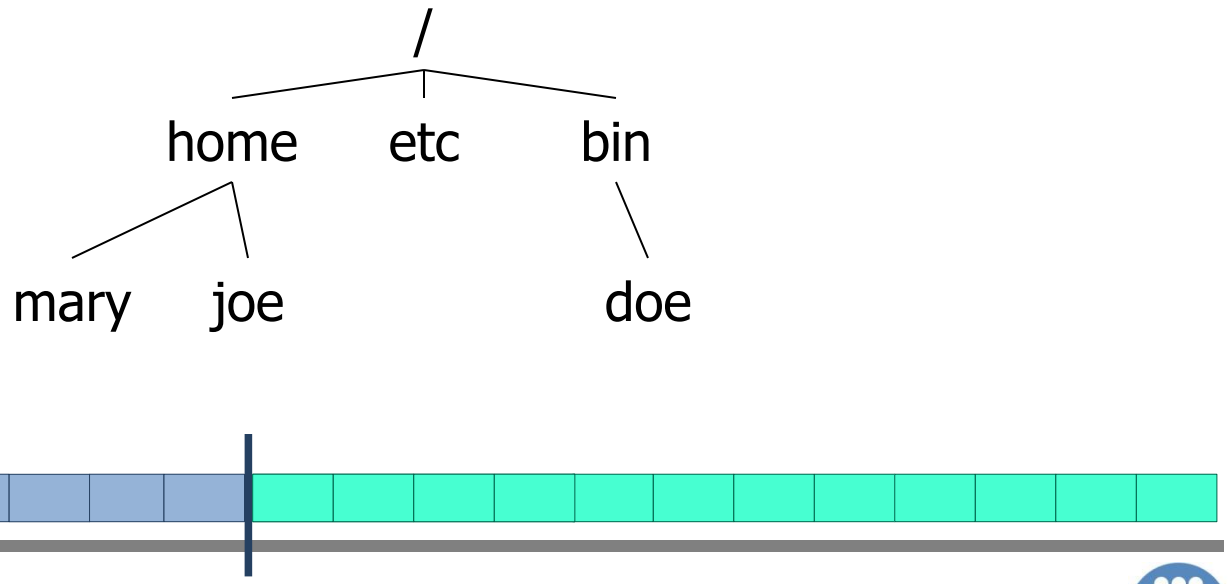
- How does a user stores/locates her data?
  - Identify data → **name**
- Map logical name and physical content
  - Users: map logical data objects to file names
  - System: manages physical content
  - File name → number → blocks
    - Ex: mydog.jpg → inode 42 → {512,513, 666, 13, …}
- Name hides implementation
  - how & where the file is stored on disk

Namespace

- How do we store more than 1 file?
  - Specially with same name
- A **namespace**: Organization of names
  - Need to create names and to lookup names
  - Represented as a labeled, directed graph with two types of nodes:
    - *Leaf node* – named entity (stores file content)
    - *Directory node* – directory table of (edge label, node id)
  - A naming path – N: <label-1, label-2, ..., label-n>
  - Path name
    - Absolute path name vs. relative path name

Directories

- Directories:
  - A user can name a file
  - Name map its location inside the FS



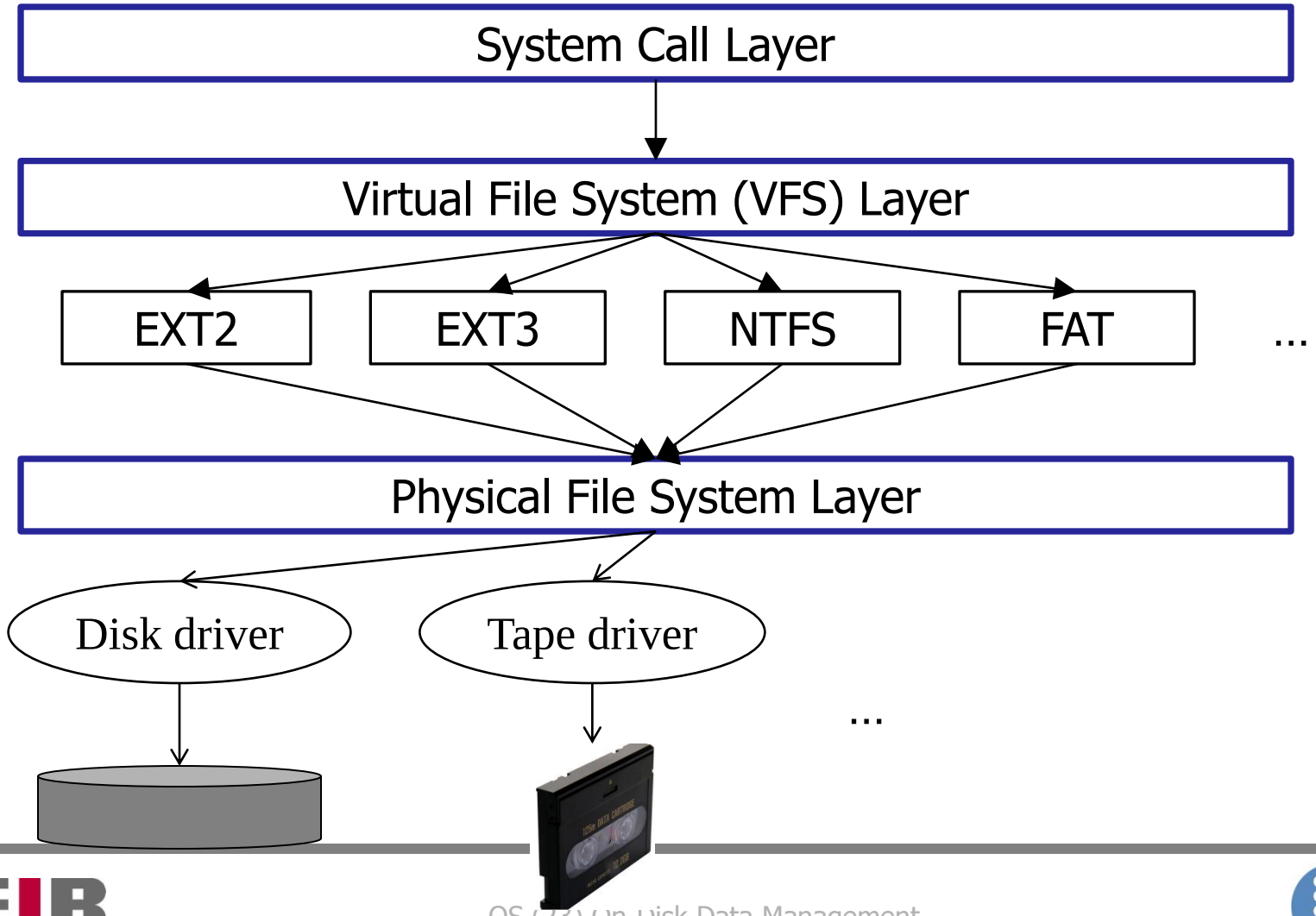
File System properties

- File System needs to solve 2 problems:
  - **Naming**: How does the user locates her data?
  - **Operations**: How does the user access her data?

Operations: VFS

- How does a user access her files?
- Virtual File System (VFS) provides uniform and transparent access to files
  - System call layer
    - Open, read, write, …
  - Virtual file system
    - Access to files independently of where or how are they stored in the device (Placement)
    - Translation file ←→ OS blocks identifiers
  - Physical file system
    - Device specific code to read/write block's content

Operations: VFS



Summary

- Desktop computer storage
  - Storage devices characteristics
  - Specific way to store data
- Desktop computer filesystem
  - OS abstraction to access data stored in a device
  - Access any device in a transparent manner
  - Naming
  - Operations

Contents

- Introduction
- Desktop Computer Storage
  - Disks
  - File System
- **WSC Storage:**
  - **Distributed File Systems**
  - Distributed Databases
  - Examples

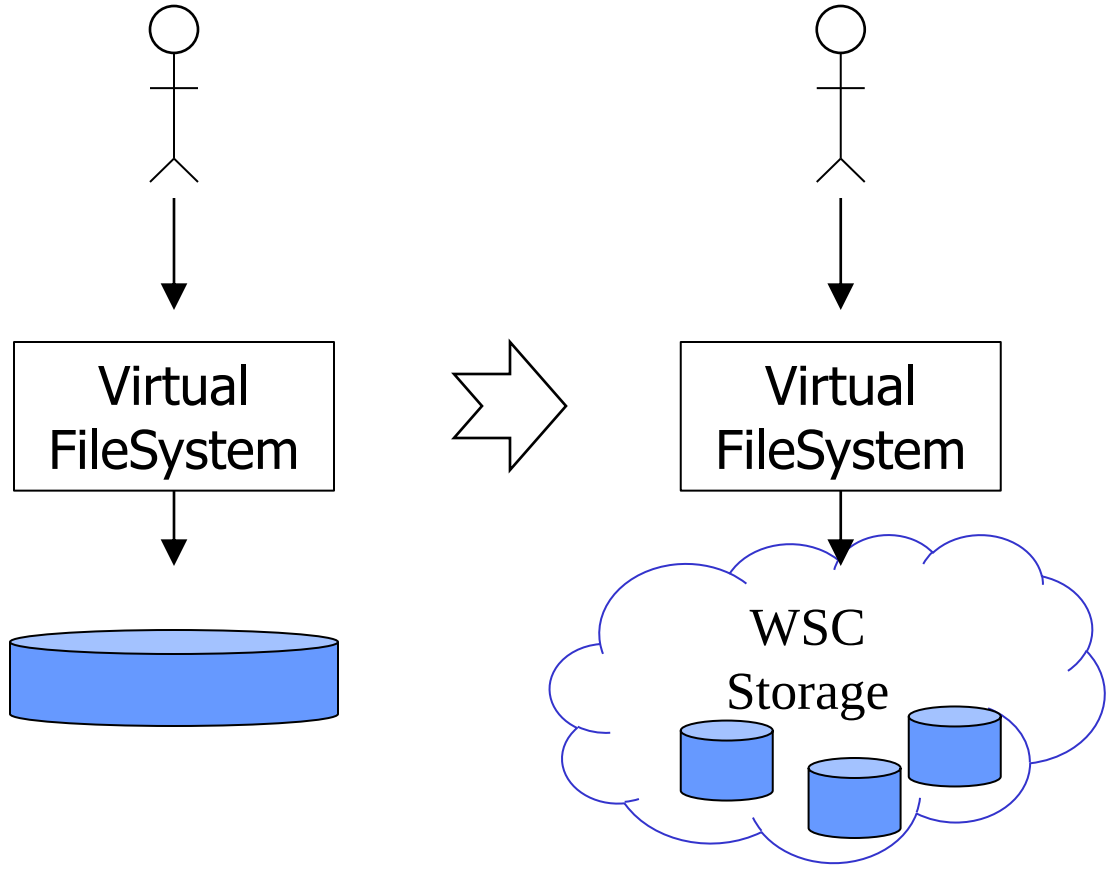
WSC

- What happens when you are away of your desktop (storage)?
  - And you need to access your data
  - User mobility (same content from anywhere)
- Let´s build a *Distributed* File System
- "The humble man's DFS"
- Transparency?
- Scalability?
- Fault-tolerance?

WSC

- Let´s build a Distributed File System
  - User mobility (same content from anywhere)
  - Transparency
  - Scalability
  - Fault tolerance
  - Share data between clients
  - Ease IT tasks (backup, update)
  - Security
  - Performance!

Storage Architecture



Changes to local filesystem

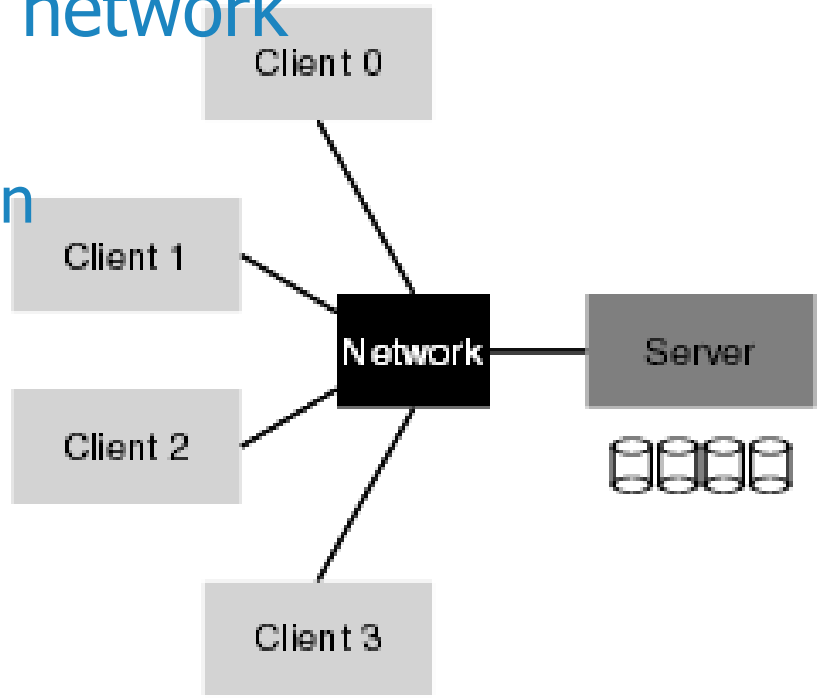
- What needs to change from local filesystem?
  - Naming
  - Operations
- Ideally, nothing should change
  - The distributed nature of the FS should be totally transparent to the user
    - Location Transparency → Hide file´s location in the network
    - Location Independence → Changing the location of a file
- But, local systems EVOLVE → new features

Distributed File Systems architecture

- Different architectures
  - **Client/Server**
  - Cluster-based

Client/Server Architecture

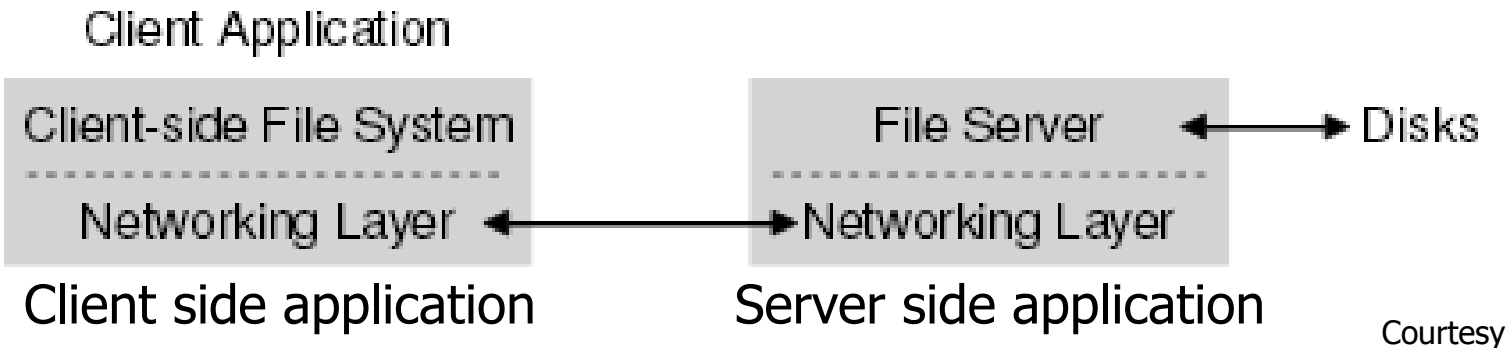
- Server has disks
- Clients configure access to server
- Clients connect through network
  - Easy sharing
  - Centralized administration
  - Security



Courtesy of [Remz18]

Client/Server Implementation

- Architecture of a simple distributed filesystem
- Client uses file system calls LOCALLY
  - From her point of view → no difference from local
- Request is forwarded to REMOTE File Server
- File Server accesess local disk
- Send back response with requested data



Courtesy of [Remz18]

DFS Access models

- How is the data accessed?
- **Remote access model**
  - Clients are offered transparent access to the FS that is managed by a remote server.
    - Remote Procedure Calls (RPCs)
- **Upload/download model**
  - Client accesses locally to a file after downloading it from the server

- Write an example of each model

Server Side

- Server may be designed as:
  - Stateful
  - Stateless
- **Stateful**
  - Store & manage files´ state between operations
  - Client requests → Update this state
    - Better performance
- **Stateless**
  - Nothing is stored at server side
  - Client stores state
  - Full requests from client with all required info
    - File, offset, bytes
    - Worse performance

Code example

- In a distributed file system, a client does:

```
char buffer[MAX];
int fd = open("foo", O_RDONLY);
read(fd, buffer, MAX);
read(fd, buffer, MAX);
…
read(fd, buffer, MAX);
close(fd);
```

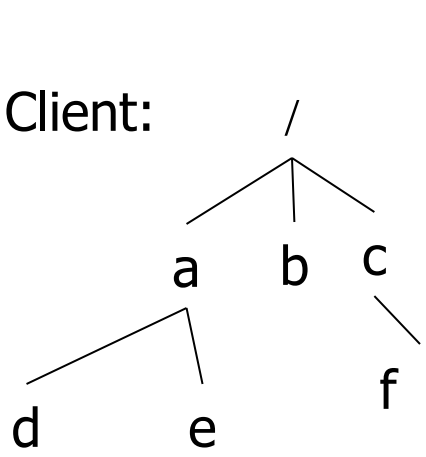
- What happens at server side?
  - Stateful
  - Stateless
- And if something crashes?

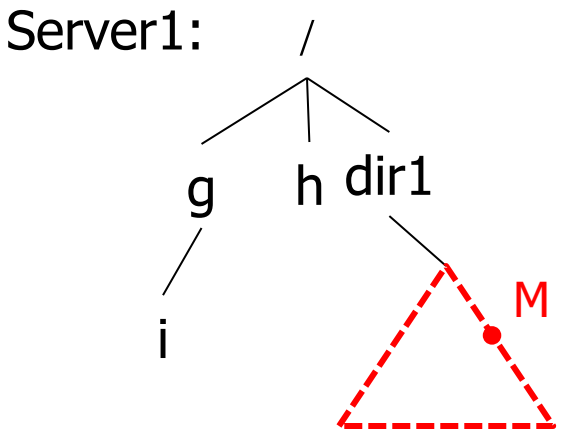
Courtesy of [Remz18]

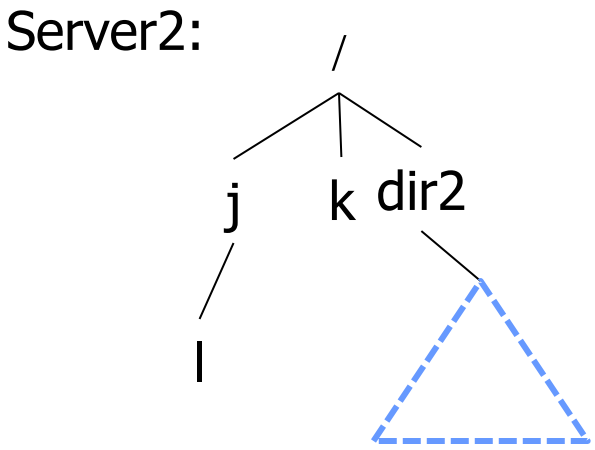
Example: NFS

- Sun Network File System (NFS)
- Client/Server file system
- Server provides a view of its local file system to clients
- Open protocol [RFC1813] specifiying the exact message formats that client and server would use to communicate
- By Design → Simple and Fast Crash recovery
  - stateless
- v4 → stateful [RFC7530]
- v4.1 → parallel access [RFC5661]→ [RFC8881]

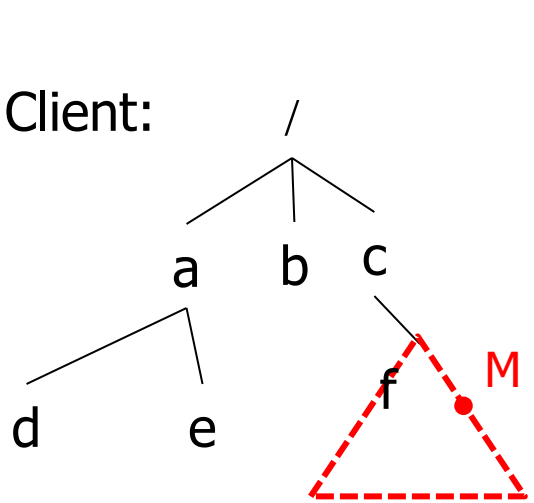
NFS (use case)

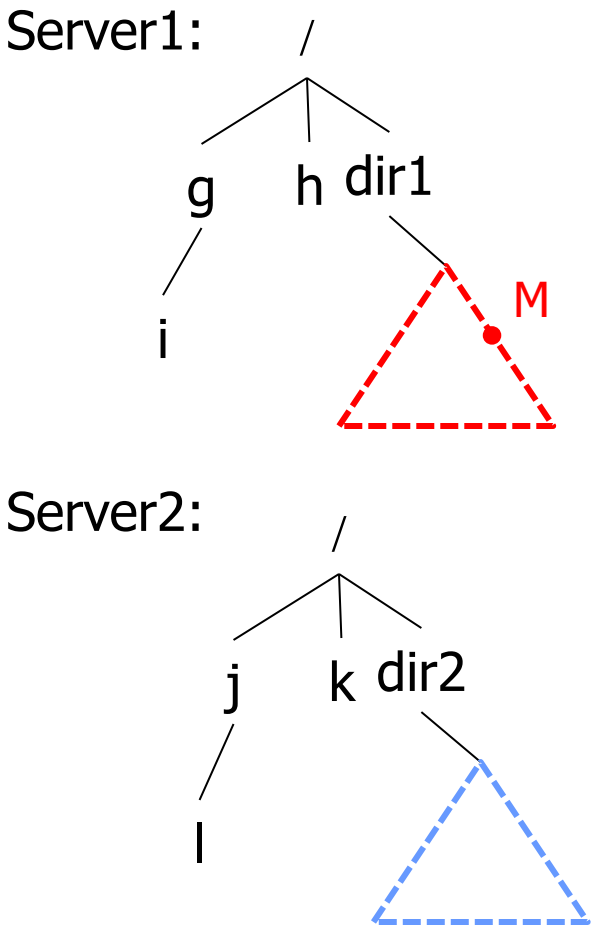




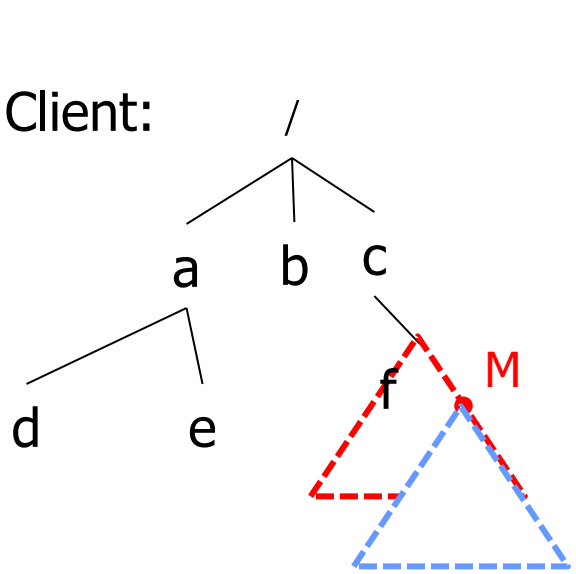


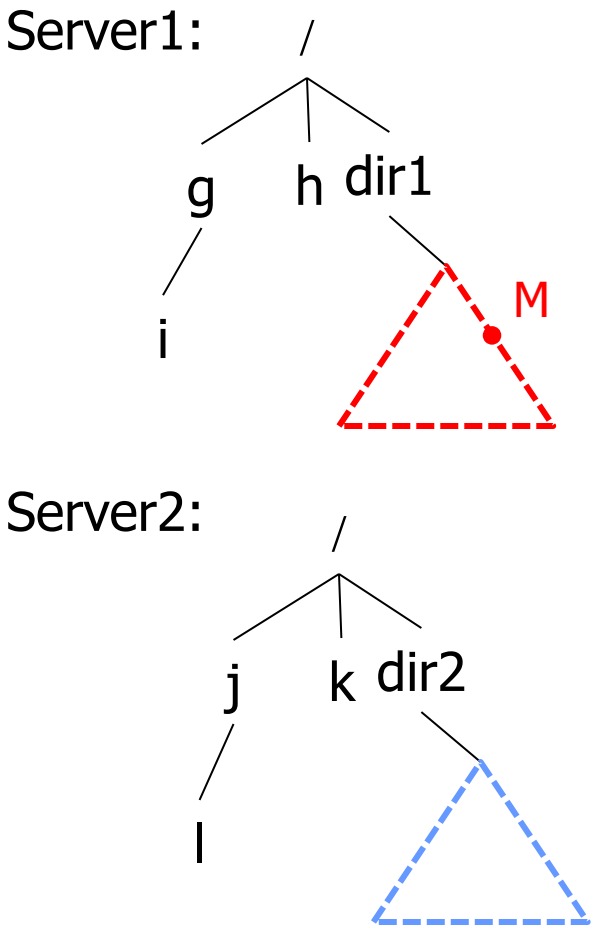
NFS (use case)



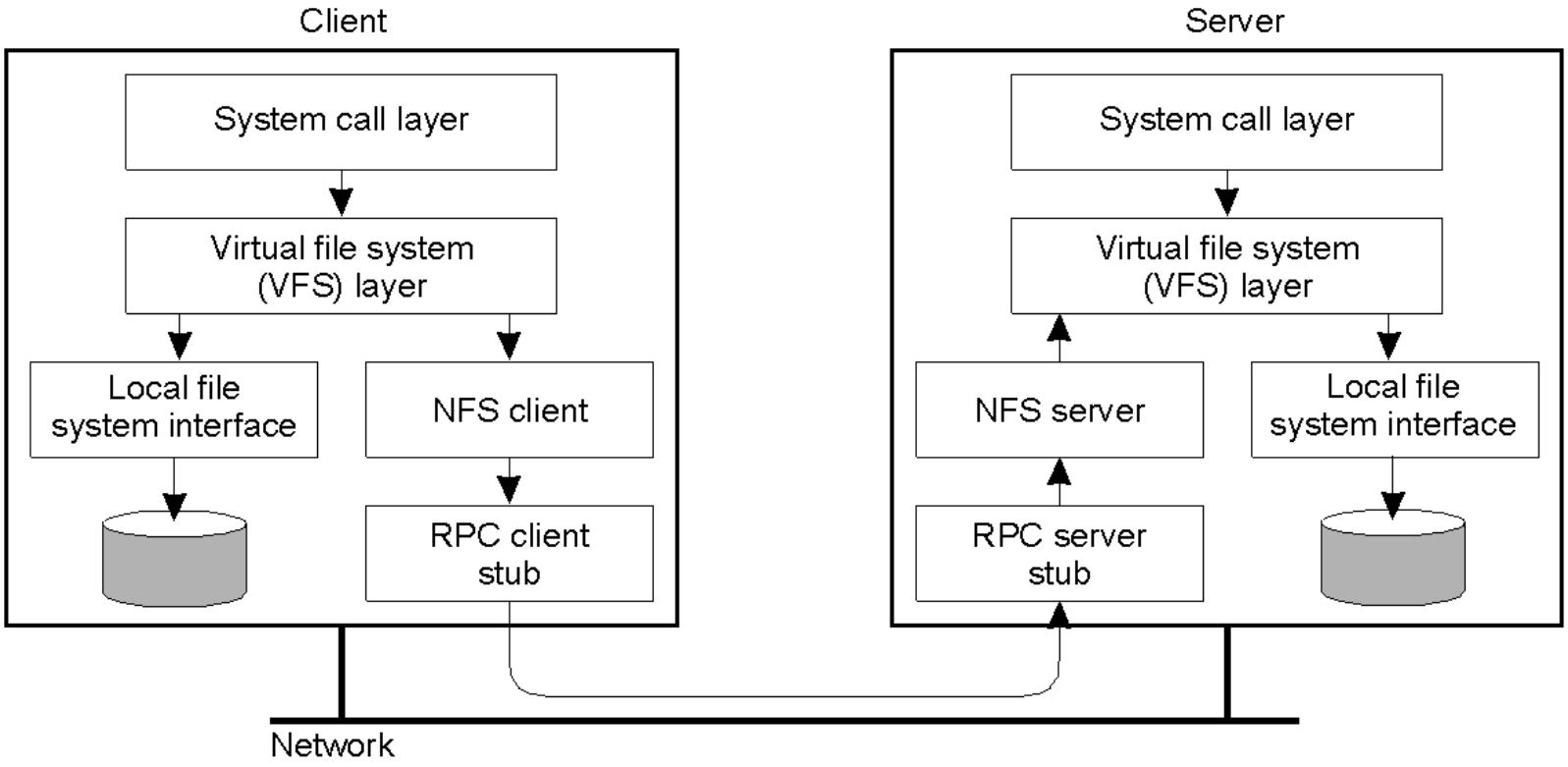


NFS (use case)





NFS Architecture



Courtesy of [Tane06]

NFS Operations

- After **mounting** a remote directory locally
- Client can:
  - Search for a file
  - Read a set of directory entries
  - Manipulate links and directories
  - Access file attributes
  - Read & Write files
- No open/close → stateless
- v4 → stateful → open&close ☺
- Problems?

Client caching

- Server is the bottleneck
- Allow clients to cache data (v4)
  - Cache data obtained from **read** operations
    - Validate cached blocks before they are used
  - **Writes** can be carried out locally
    - Modified blocks are marked as 'dirty'
    - **Closing** the file implies flushing modifications to server
  - File can remain cached after closing it
    - Data must be revalidated when file is reopened
- Validation through timestamps
- Problems?

Oh, No! Consistency Problems again!

- On single machine, when read follows write
  - value returned by read is value just written
- In distributed system with caching
  - stale values may be returned!
- 4 approaches to deal with shared files in distributed systems

Method	Comment
UNIX semantics	Every operation on a file is instantly visible to all processes
Session semantics	No changes are visible to other processes until the file is closed
Immutable files	No updates are possible; simplifies sharing and replication
Transactions	All changes occur atomically (BEGIN/END_TRANSACTION)

Distributed File Systems architecture

- Different architectures
  - Client/Server
  - **Cluster-based**

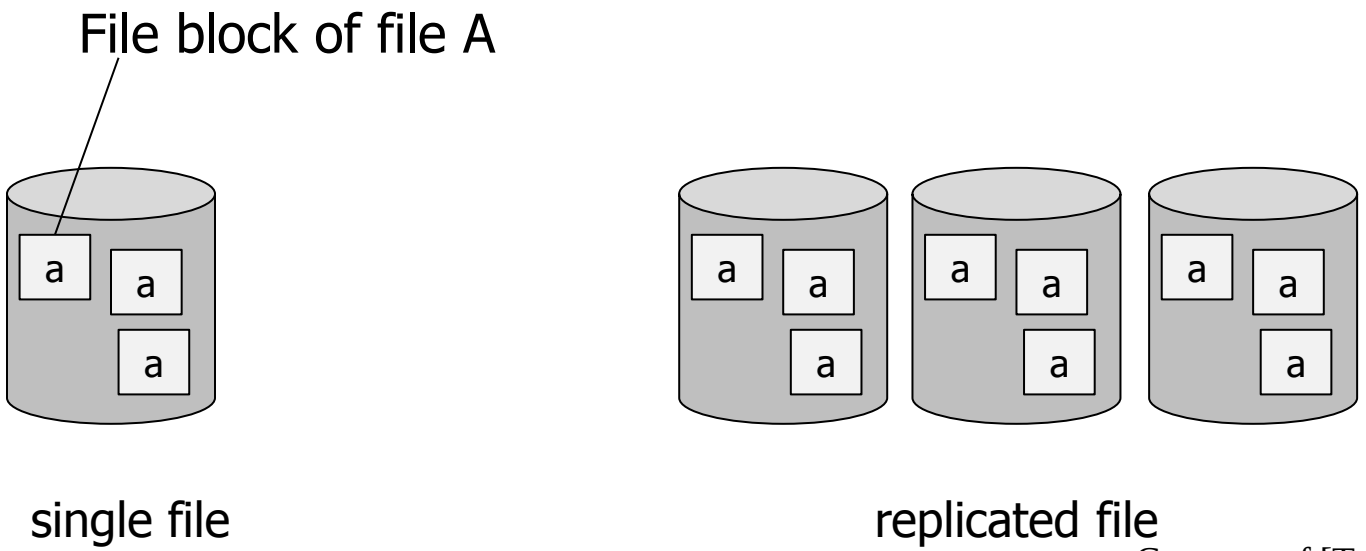
Cluster-based architecture

- Server is the bottleneck
- What are the limiting factors?
- How do we improve performance in server clusters?
  - Replicate
  - Distribute/Partition
- Paper: GFS Google File System

[Ghema03] Ghemawat, S., Gobioff, H., Leung, S. T., The Google file system. ACM SIGOPS Operating Systems Review, Volume 37, Issue 5, pp. 29-43, 2003.

Replication

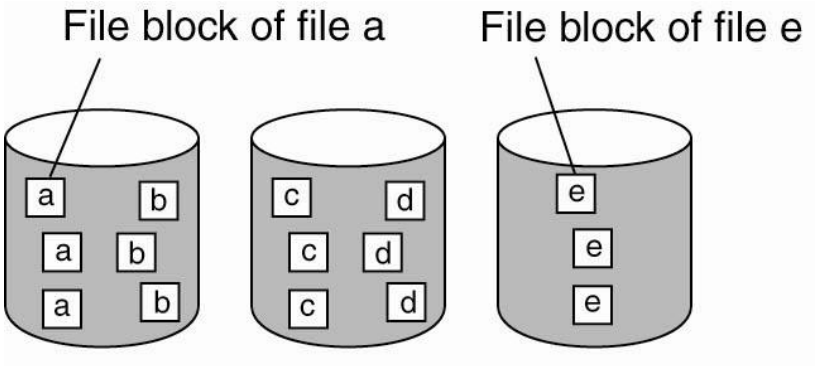
- Single file is accessed by multiple clients
  - Replicate the file through multiple servers
    - (like in memory)
    - (~RAID1)



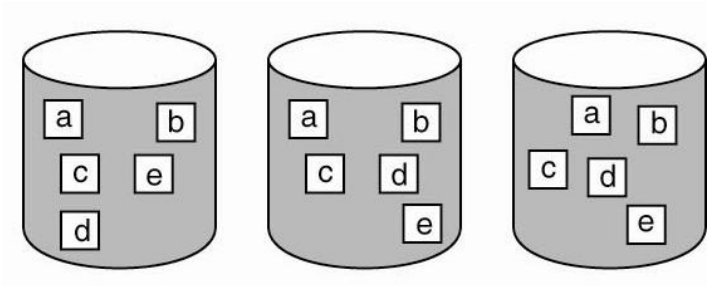
Courtesy of [Tane06]

Distribution/Partition

- File-stripping techniques by which files can be fetched in parallel
  - Single file is distributed across multiple servers
  - (~RAID0)



whole-file distribution

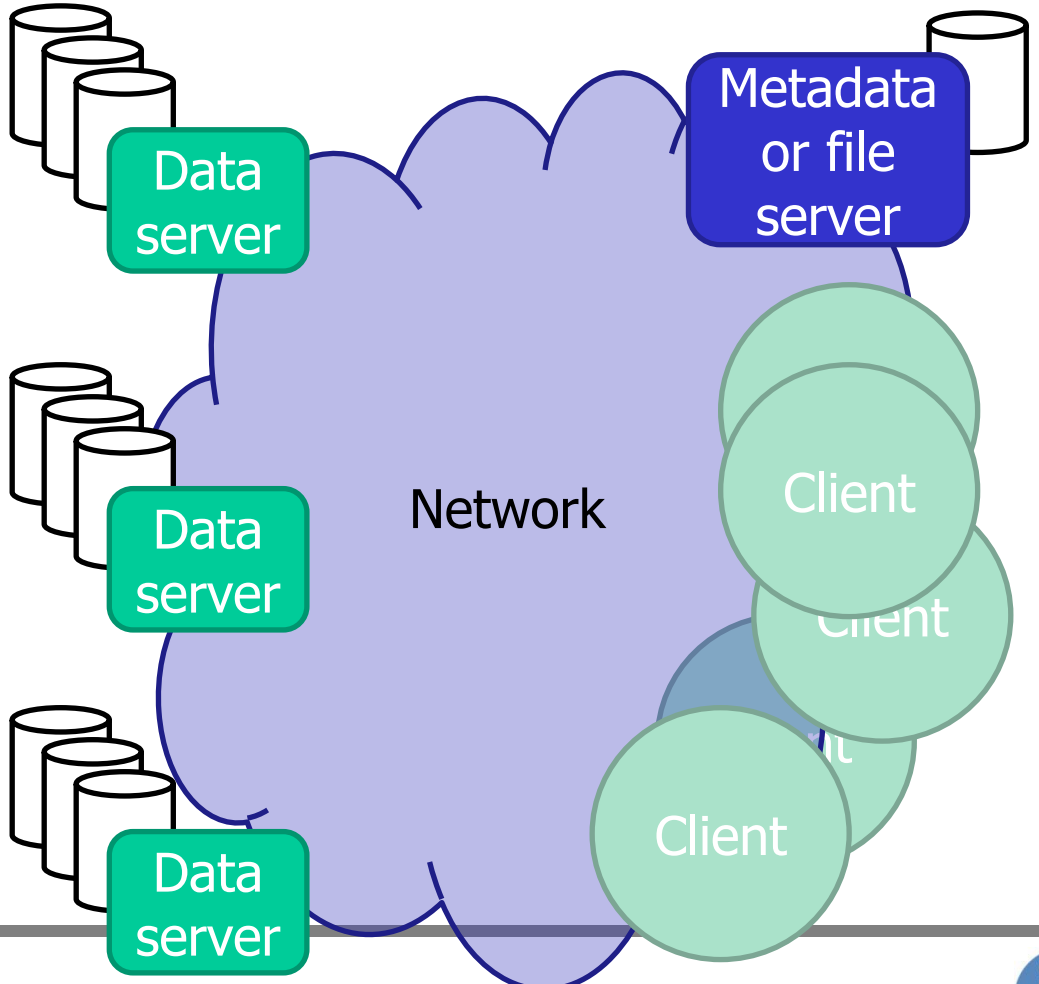


file-stripped system

Courtesy of [Tane06]

- How does the client know where to find the files and its parts with these improvements?
  - Metadata server
    - (and a load balancer if needed)

Basic cluster-based file-system architecture



Consistency Implementation

- Immutable
  - No algorithm needed

  No updates are possible
- Sequential/UNIX semantics
  - Avoid replication
    - ➔ no algorithm needed
      - Viable?
  - Avoid write caching
    - ➔ no algorithm needed
      - slow?
  - Only cache on server
    - ➔ no algorithms needed
      - bottleneck?
  - **Coherence algorithm**

  Every operation on a file is instantly visible to all processes
- Session and transaction
  - **Coherence algorithm**

  No changes are visible to other processes until the file is closed

Distributed File Systems Summary

- Changes to Local File systems
  - Naming & Operations
- Architectures
  - Client/server
    - Different access models
      - Remote Acces vs Upload/Download
    - Different server implementation
      - Stateful & Stateless
    - Performance → replicas → consistency problems
  - Cluster based
    - Replication & Distribution
    - Consistency problems

Contents

- Introduction
- Desktop Computer Storage
  - Disks
  - File System
- WSC Storage:
  - Distributed File Systems
  - **Distributed Databases**
  - Examples

How do users access data?

- Files
- Higher levels (remember the memory case?)
- Structured
- Not structured

How do users access data?

- Files
- Higher levels (remember the memory case?)
- **Structured**
- Not structured

DDBB

- Data stored in tables
- Structured tables
- Each row (data item) in a table has a primary key unique with that table
- Queried using SQL
  - Which at runtime, translates to file accesses
- Support Joins

DDBB Example

Employees table

Id	Name	Email	Phone	JobID
1	John	j@a.es	666666	3
2	Mary	m@hotmail.es	777777	2
3	Doe	d@msn.com	424242	1

Jobs table

Id	Name	Salary
1	Slave	1000
2	CTO	20000
3	CEO	50000

SQL syntax Creation

```
CREATE TABLE Persons (
    PersonID int NOT NULL,
    Name varchar(255),
    Email varchar(255),
    Phone varchar(255),
    JobID int,
    PRIMARY KEY(PersonID)
    FOREIGN KEY(JobID) REFERENCES Jobs(JodID)
);
```

```
CREATE TABLE Jobs (
    JobID int NOT NULL,
    Name varchar(255),
    Salary int,
    PRIMARY KEY(JobID)
);
```

SQL syntax insert & select & update

```
INSERT INTO Employees (PersonID, Name, Email, Phone, JobID)
VALUES (1, 'John', 'j@a.es', '666666', 3);
```

```
SELECT * FROM Employees;
```

```
UPDATE Employees SET Email='johny@a.es' WHERE PersonID=1;
```

But today´s data…

- Large and unstructured
- Lots of random reads & writes
- Sometimes huge number of writes
- Foreign keys rarely used
- Joins infrequent

Needs…

- Speed
- Avoid *Single Point of Failure* (SPoF)
- Low TCO (Total cost of operation)
- Fewer system administrators
- Incremental Scalability
- Scale out, not up
  - What?

Scale up vs Scale out

- **Scale up** = grow your cluster capacity by replacing with more powerful machines
  - Traditional approach
  - Not cost-effective, as you're buying above the sweet spot on the price curve
  - And you need to replace machines often
- **Scale out** = incrementally grow your cluster capacity by adding more COTS machines (Components Off the Shelf)
  - Cheaper
  - Over a long duration, phase in a few newer (faster) machines as you phase out a few older machines
  - Used by most companies who run datacenters and clouds today

How do users access data?

- Files
- Higher levels (remember the memory case?)
- Structured
- **Not structured**

Remember CAP

- Consistency
- Availability
  - 1 ms latency increase → $6M/year loss
  - SLA´s deal with client´s latencies
- Partition tolerance
- 3 at a time are not possible
- → Favor 2

Key-Value / NoSQL Data Model

- NoSQL → "Not only SQL"
- Need API ops: get(key) + put(key, value)
  - And some extended operations (CQL@Cassandra)
- Tables
  - Different names:"Column families" Cassandra, "Table" Hbase, "Collection" MongoDB
  - Like RDBMS, but…
    - Not structured
    - Missing columns from rows
  - No support for joins or foreign keys
  - Can have index like RDBMs
- BASE vs ACID

DDBB Example

Employees table

Id	Name	Email	Phone	JobID
1	John	j@a.es	666666	3
2	Mary	m@hotmail.es	777777	2
3	Doe	d@msn.com	424242	1

Jobs table

Id	Name	Salary
1	Slave	1000
2	CTO	20000
3	CEO	50000

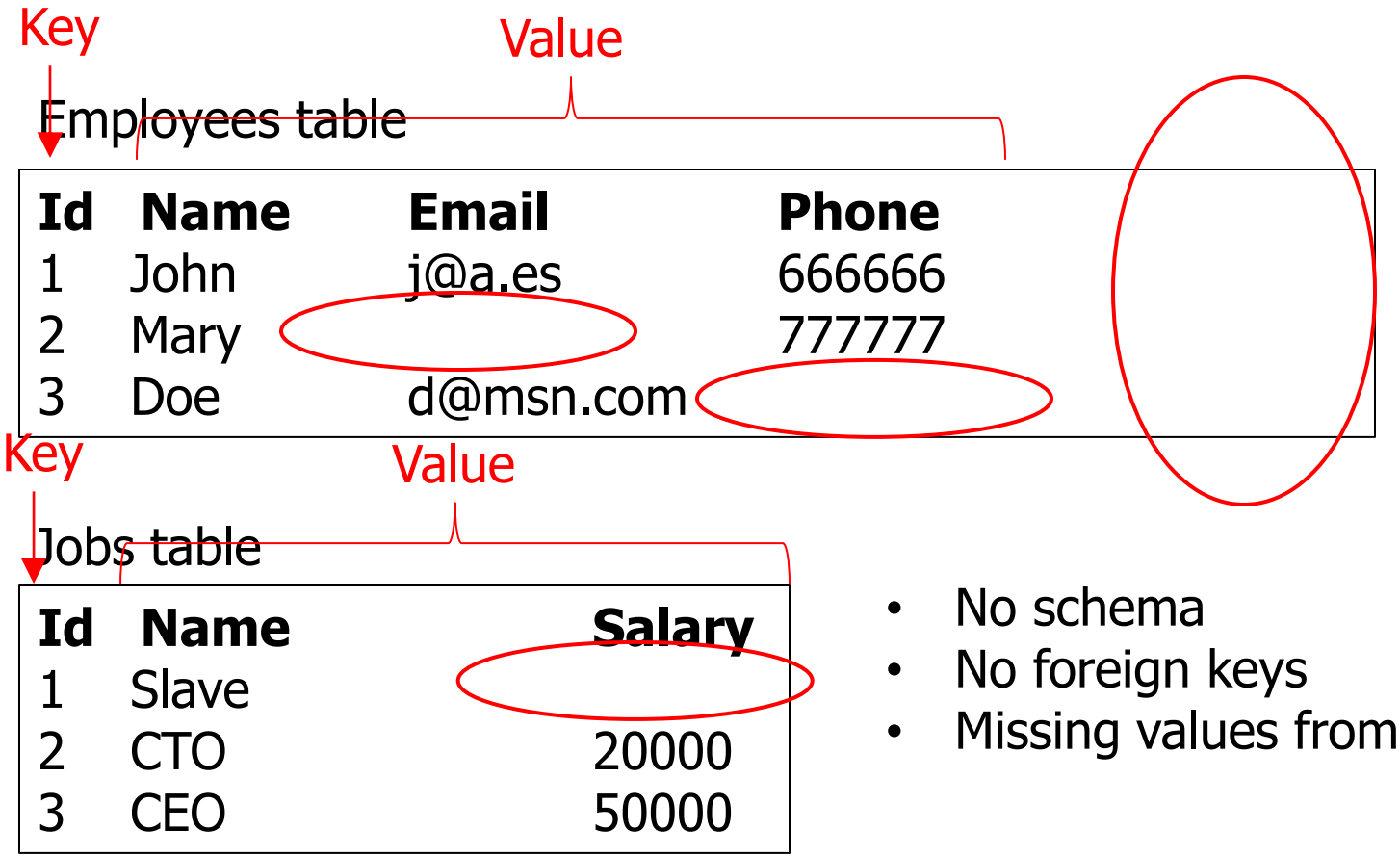
CQL syntax creation

```
CREATE KEYSPACE IF NOT EXISTS store WITH REPLICATION =
{ 'class' : 'SimpleStrategy',
'replication_factor' : '1'
};

CREATE TABLE IF NOT EXISTS store.shopping_cart (
    userid text PRIMARY KEY,
    item_count int,
    last_update_timestamp timestamp
);

INSERT INTO store.shopping_cart
    (userid, item_count, last_update_timestamp)
    VALUES ('9876', 2, toTimeStamp(now()));
INSERT INTO store.shopping_cart
    (userid, item_count, last_update_timestamp)
    VALUES ('1234', 5, toTimeStamp(now()));
```

Key-Value Example



- No schema
- No foreign keys
- Missing values from rows

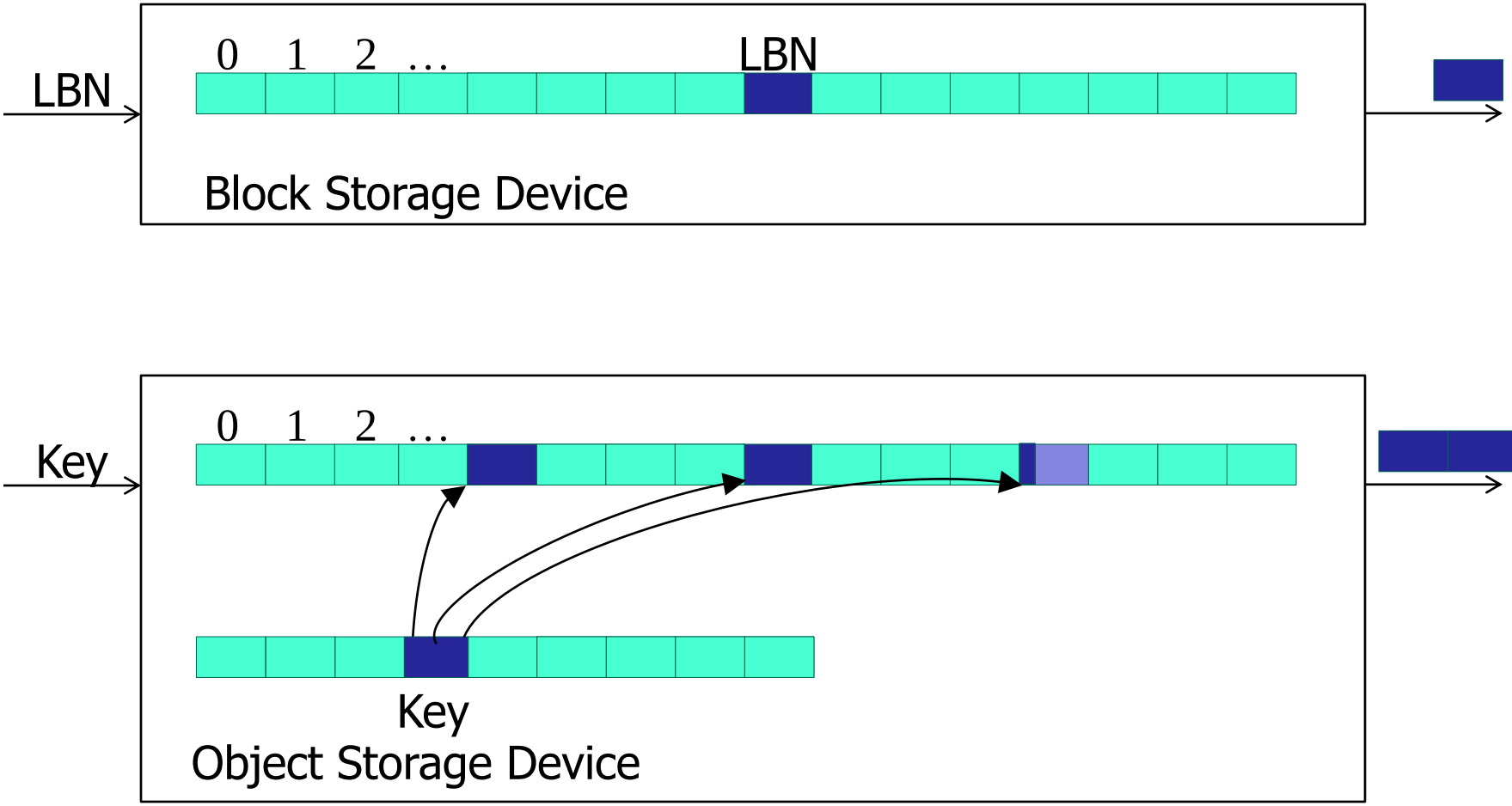
Storing DDBB data

- How do we store these tables in the devices?
  - Structured data (fixed size) fit well in a block based filesystem
  - But the new model (variable size) does not fit too well
- A more flexible system is needed
  - Object file system

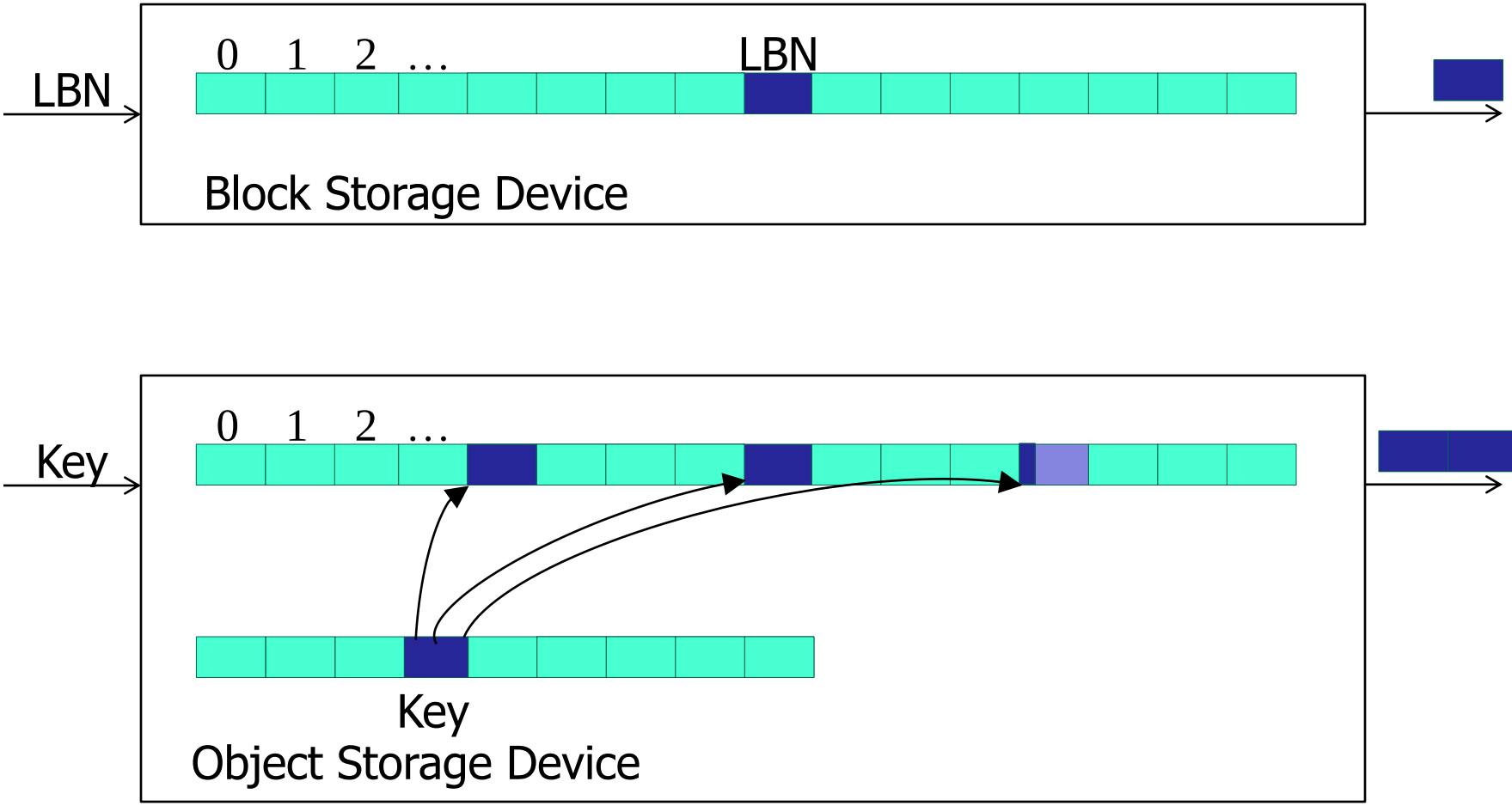
Block vs Object file system

- Block storage device
  - Data stored in **fixed-size** blocks
  - Identified by positive integer starting from 0
    - Logical Block Number (LBN)
  - LBN used for data retrieval
  - Block-based file system → store files
- Object storage device
  - Data is a single object
  - Key & Value
  - Key → Positive integer (64bits)
  - Value → **Arbitrary size**
  - Attributes
  - Object-based file system → store files

Block/Object storage device

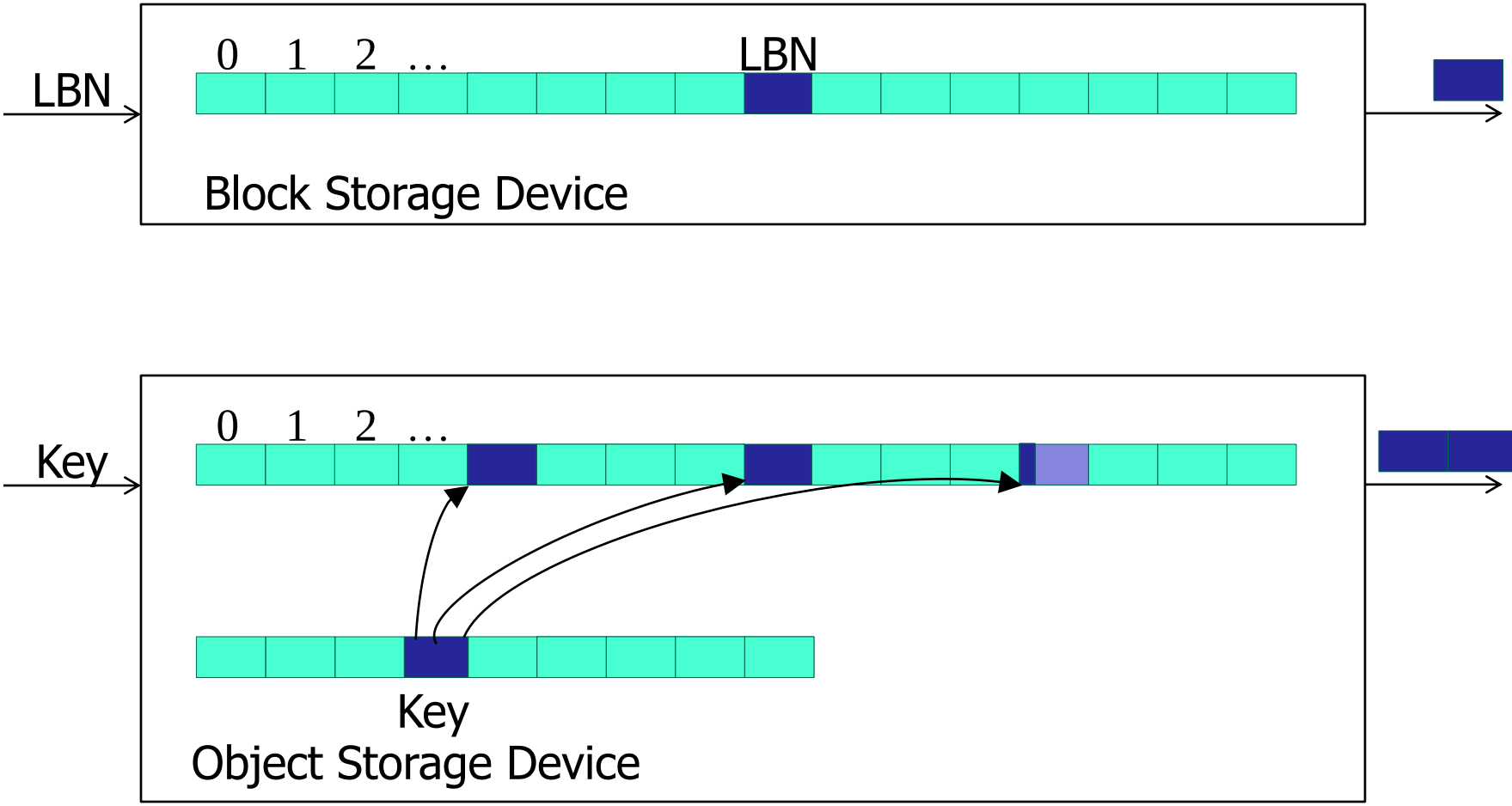


Block/Object storage device



How do you build a file system on top of this?

Block/Object storage device

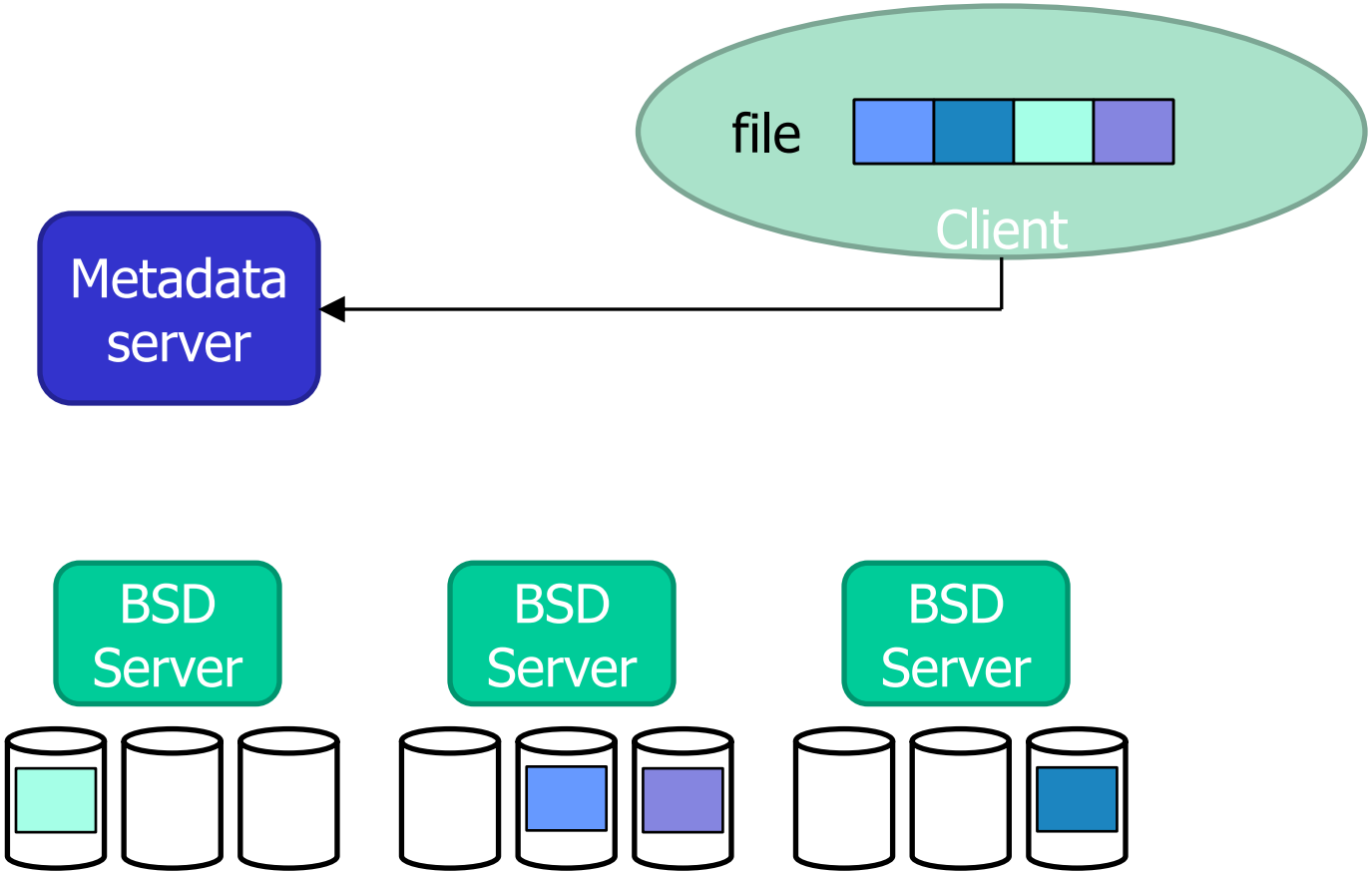


How do you build a file system on top of this? And a distributed one?

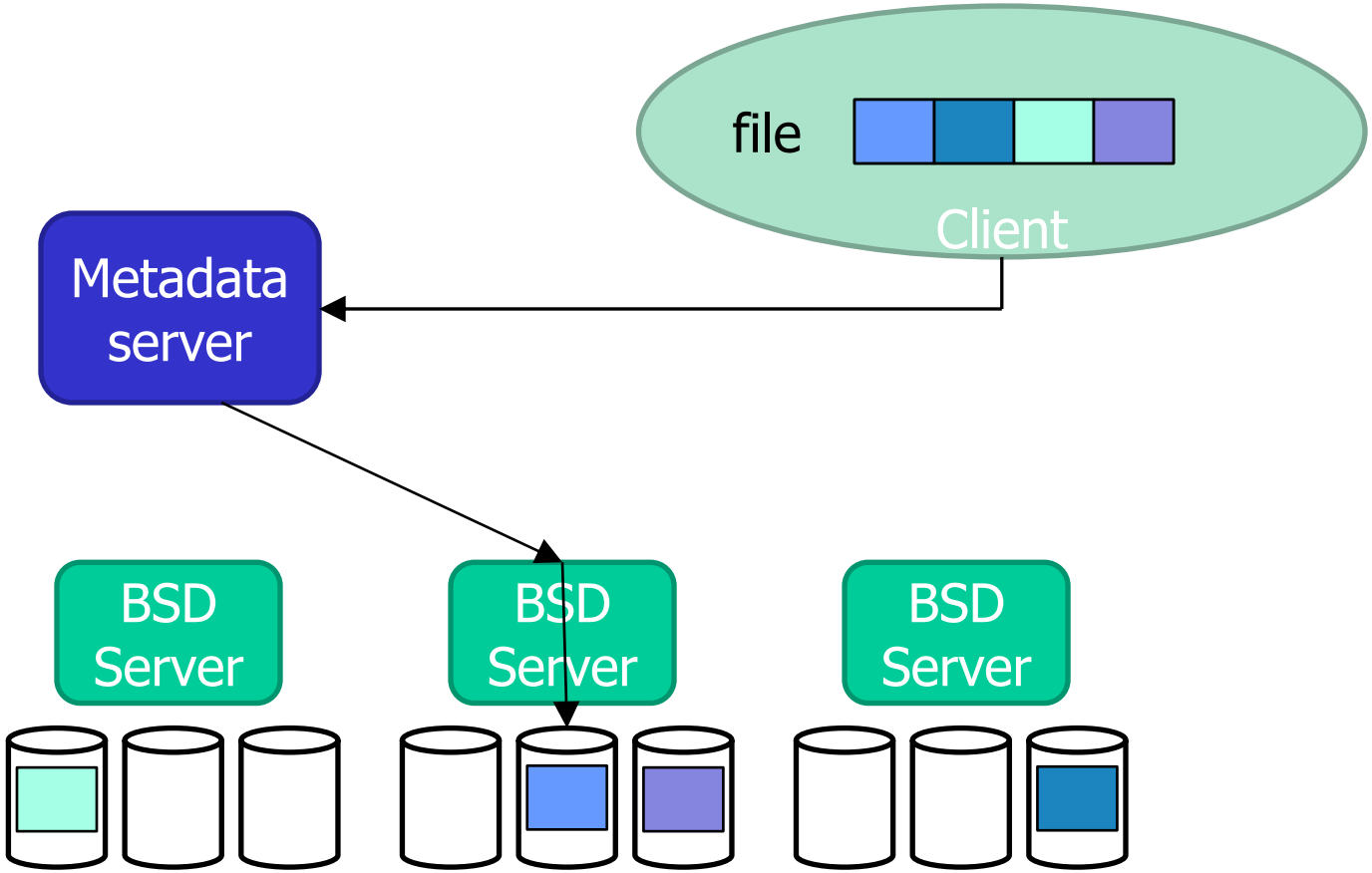
Block vs Object file system

- Block-based distributed file systems
  - Metadata servers are responsible for
    - Metadata
    - Where the data is located in the disks (disk and block)
- Object-based distributed file system
  - Metadata servers are responsible for
    - Metadata (about object)
  - Object storage devices are responsible for
    - Where the data is located in the disks (disk and block)
  - Object storage devices cooperate and reduce bottlenecks

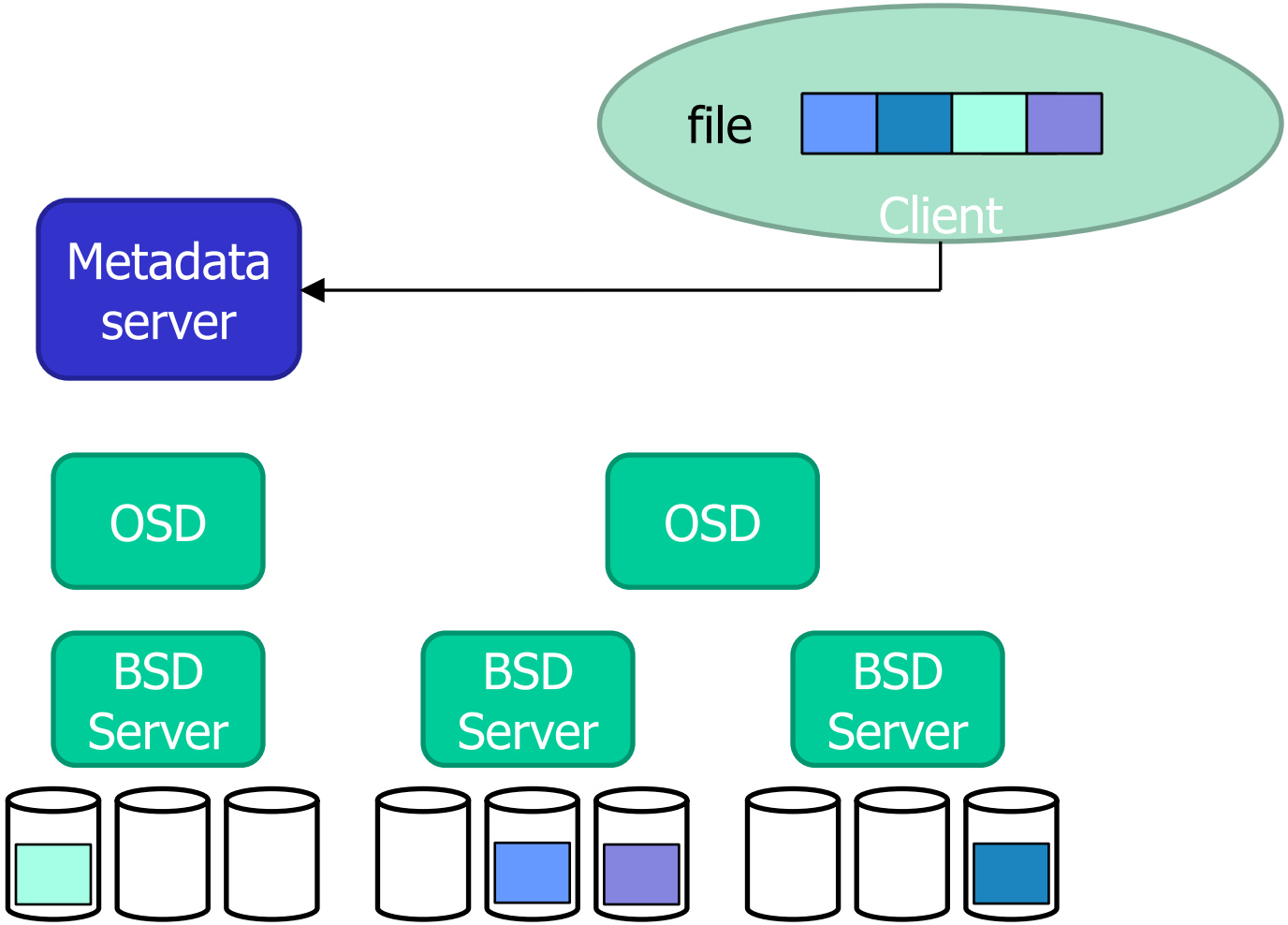
Block-based distributed file system



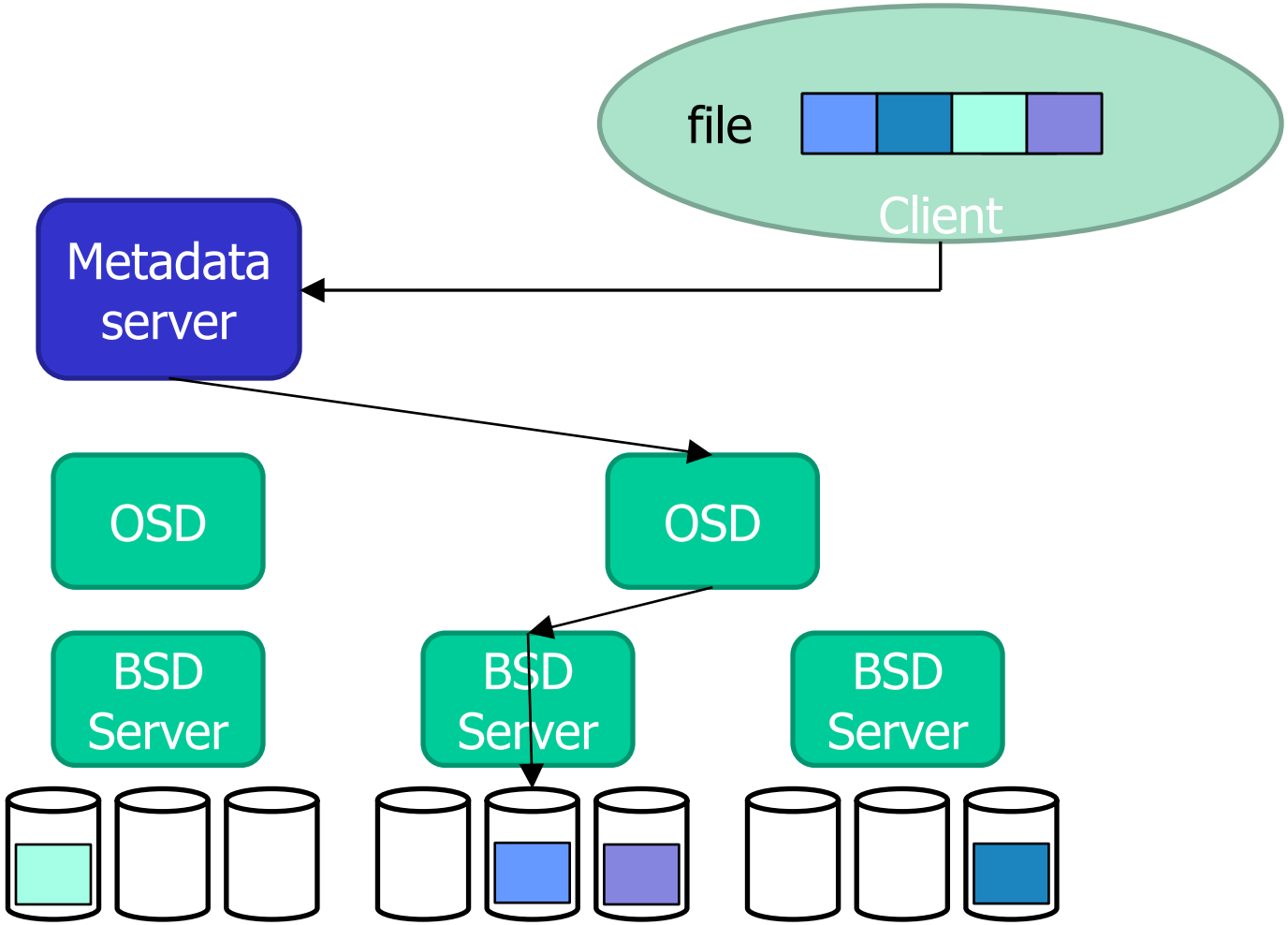
Block-based distributed file system



Object-based distributed file system



Object-based distributed file system



Block-storage file system

- Steps to create a file
  - Client access the metadata server
    - Allocates an i-node
- Steps to access the data
  - Client access the metadata server
    - Sends file name and offset within file
  - Metadata server replies with disk and block
    - Or sends the block directly to the client

Object-storage file system

- Steps to create a file
  - Client access the metadata server
    - Allocates an i-node
  - Contacts OSDs to create objects for this file
- Steps to access the data
  - Client access the metadata server
    - Sends file name
  - Metadata server replies with a list of OSDs
  - Client contacts OSDs with file and offset within file
  - OSD replies with data

Object-storage filesystem advantages

- Decouples Metadata from Data location
- Data can be moved inside OSDs without notification
  - Enables optimizations
- Metadata operations do not interfere with data operations
  - Different servers
  - Clients access directly both
    - Metadata servers
    - OSDs

Distributed databases summary

- Change in access data
  - Files → database
- SQL → NoSQL
  - Scalability
    - Replication & Distribution
- Internal implementation of DDBB
  - Files
  - Objects
- Mapping to underlying filesystems
  - Block based
  - Object based

Contents

- Introduction
- Desktop Computer Storage
  - Disks
  - File System
- WSC Storage:
  - Distributed File Systems
  - Distributed Databases
  - **Examples**

Filesystem Use cases

- GPFS
- Dropbox
- Googlefs
- HDFS

Filesystem Use cases

- **GPFS**
- Dropbox
- Googlefs
- HDFS

IBM General Parallel File System (GPFS)

- High performance clustered file system
- **Posix Compliant**
- Directory + Metadata distributed across file system
- File split in blocks of 1MiB
- No replication
- Tolerates network partitions
  - The largest partition remains live.
- Used in **HPC**
- [Schm02]

Filesystem Use cases

- GPFS
- **Dropbox**
- Googlefs
- HDFS

Dropbox

http://www.cs.rutgers.edu/~pxk/417/notes/content/15-dfs-slides.pdf

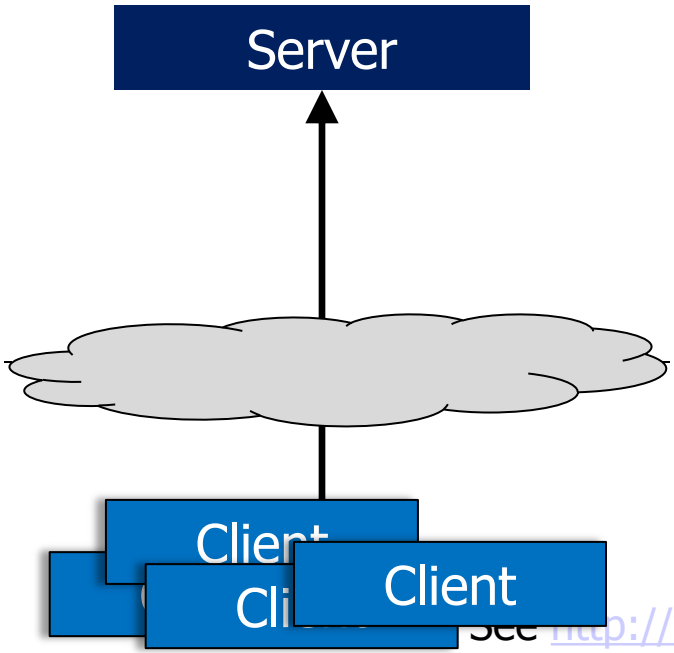
- Client runs on desktop
- Uploads any changes made within a dropbox folder
- Huge scale
  - 100+ million users syncing 1 billion files per day
- Design
  - Small client that doesn't take a lot of resources
  - Expect possibility of low bandwidth to user
  - Scalable back-end architecture
  - 99%+ of code written in Python
    - ⇒ server software migrated to Go in 2013

Dropbox

- Typically read to write ratios are really high
- But dropbox is 1:1

Dropbox Architecture evolution: v1

- one server: web server, app server, mySQL database & sync server

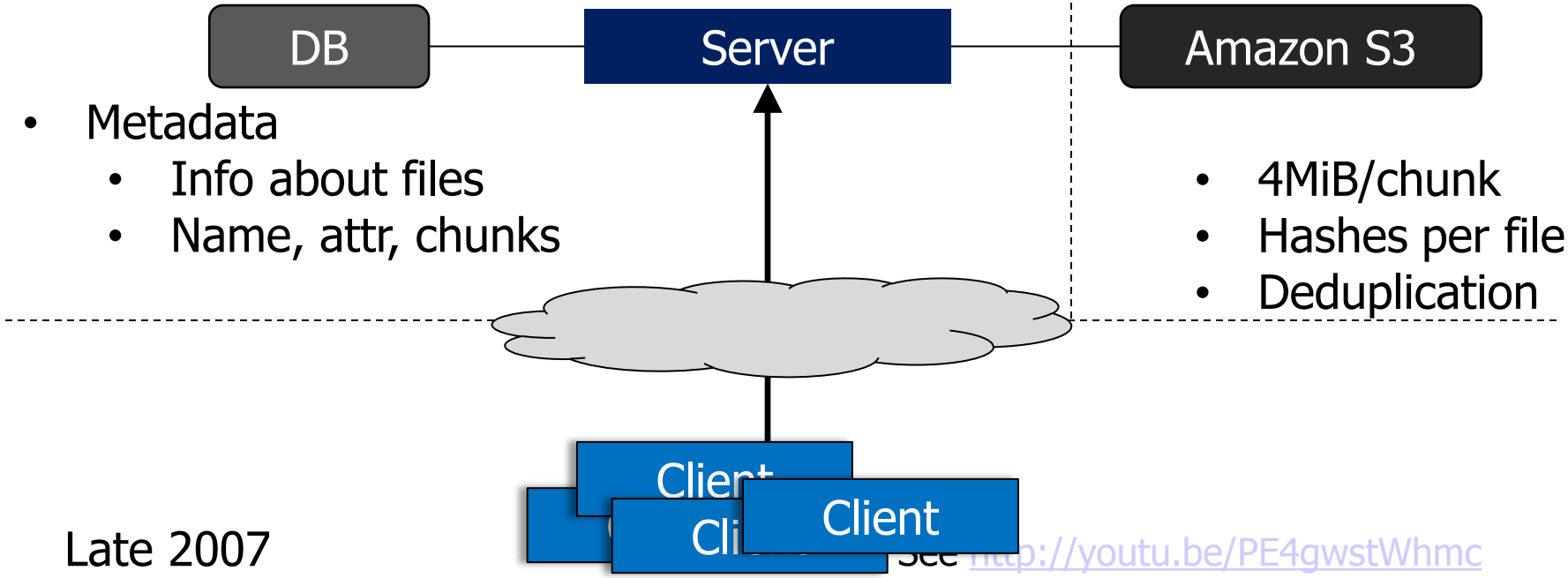


2007

See http://youtu.be/PE4gwstWhmc

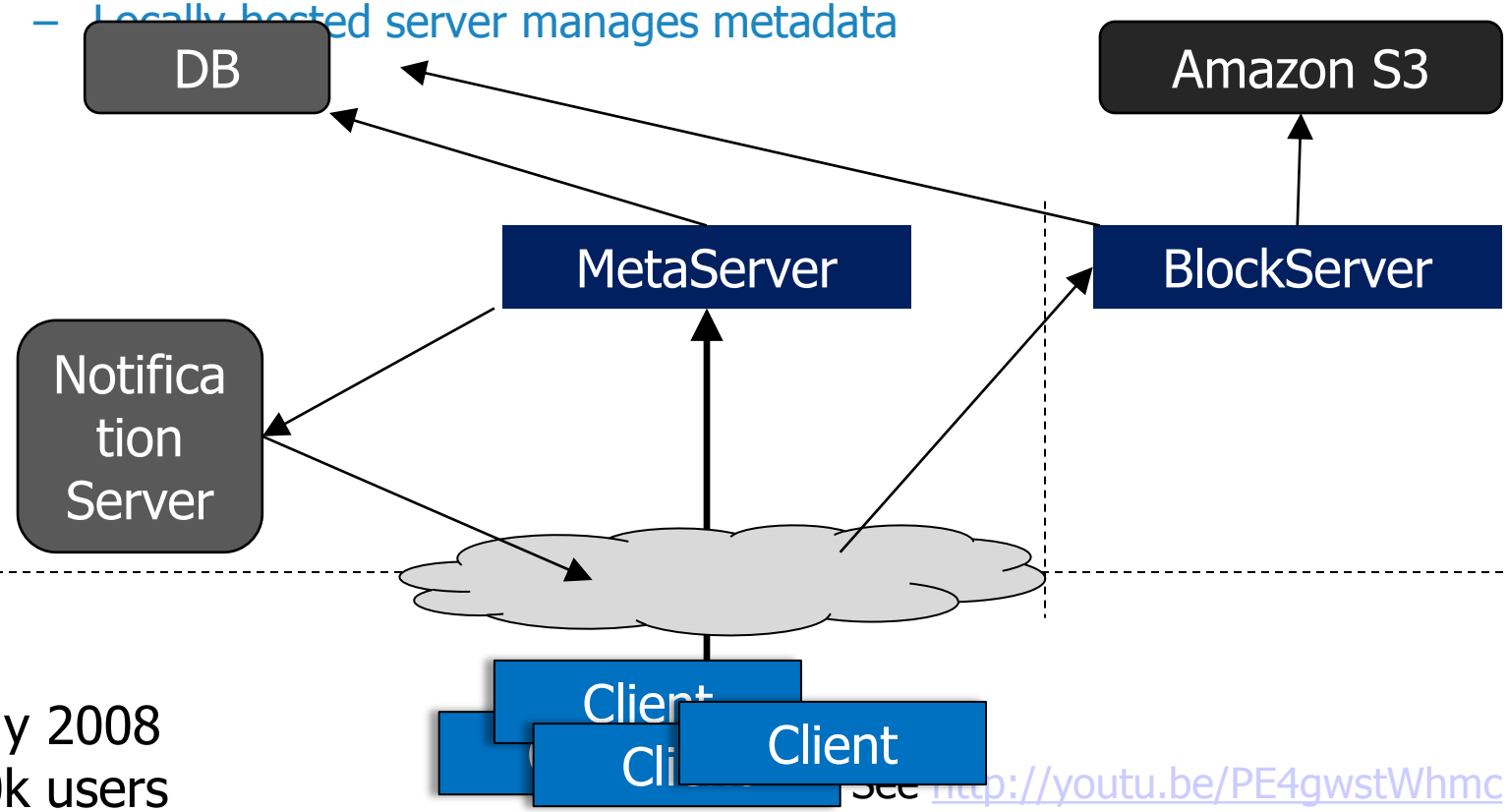
Dropbox Architecture evolution: v2

- Server run out of space → Amazon S3
- Overloaded server → separate DB
- Clients polling server about new updates



Dropbox Architecture evolution: v3

- Moved from polling to notification server
- Split server in two
  - Amazon hosts file content & uploads
  - Locally hosted server manages metadata

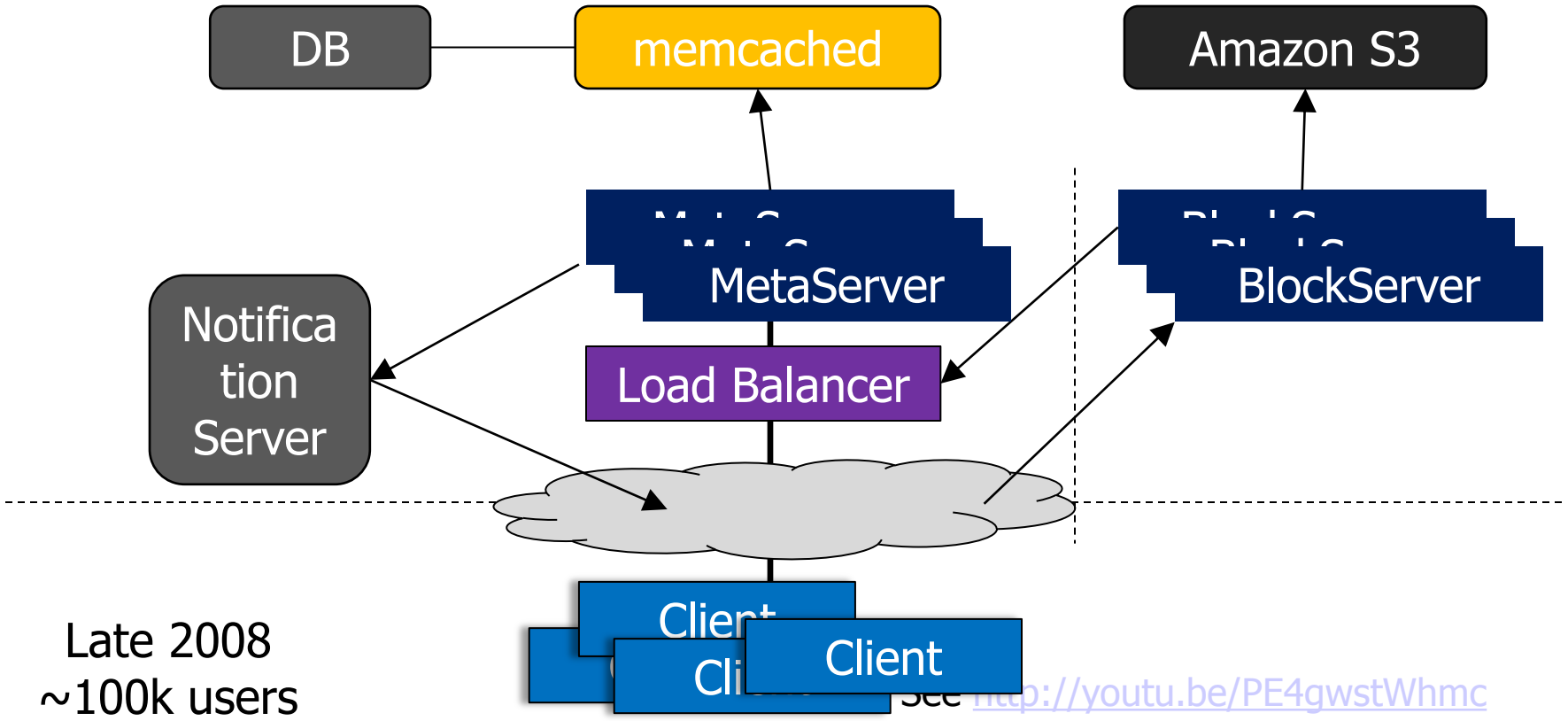


Early 2008
~50k users

See http://youtu.be/PE4gwstWhmc

Dropbox Architecture evolution: v4

- Added more metaservers & blockservers
- Blockserver do not access DB directly, uses RPC to metaservers
- Add memory cache in front of DB

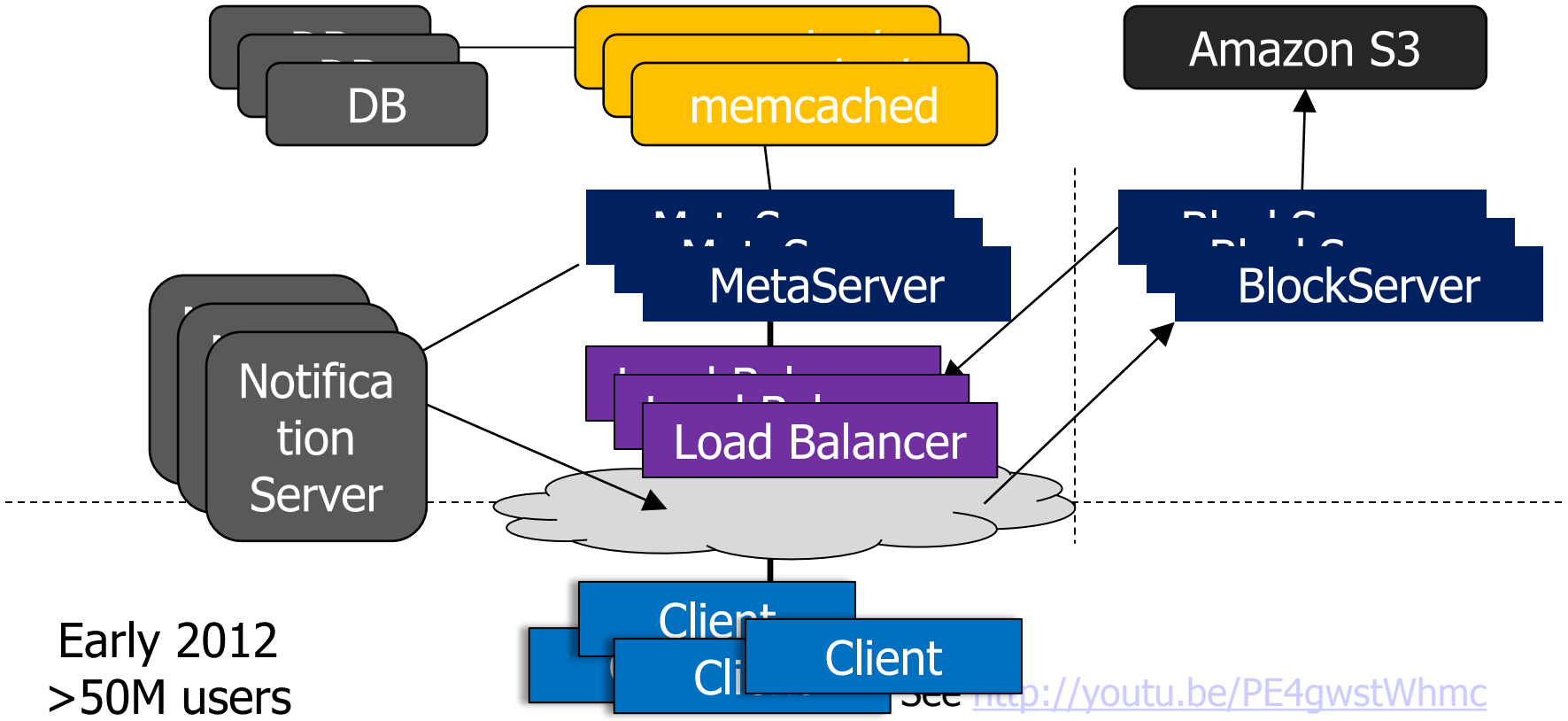


Late 2008
~100k users

See http://youtu.be/PE4gwstWhmc

Dropbox Architecture evolution: v5

- Clients have to connect before getting notifications
- Add 2-level hierarchy to notification servers



Early 2012
>50M users

See http://youtu.be/PE4gwstWhmc

Dropbox Architecture evolution:

- ~40PB in 2012
- 12x growth till 2016!
- What do you think the next step was?
- "Magic Pocket" →custom-built infrastructure
  - Hardware and Software!
  - Further reading:
    - https://blogs.dropbox.com/tech/2016/03/magic-pocket-infrastructure/
    - https://blogs.dropbox.com/tech/2016/05/inside-the-magic-pocket/

Filesystem Use cases

- GPFS
- Dropbox
- **Googlefs**
- HDFS

Google File System [Ghem03]

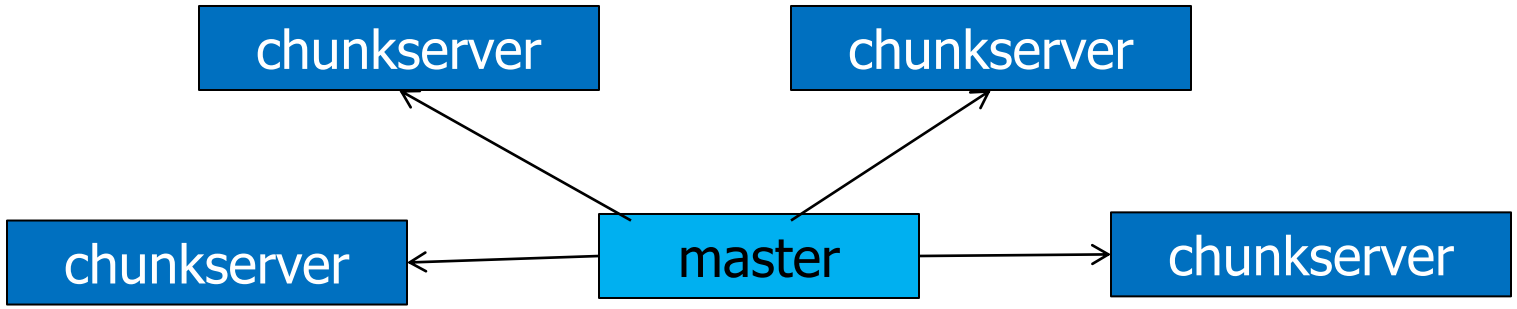
- A distributed file system developed by Google to provide efficient, reliable access to data using large clusters of **commodity hardware**.
- Design:
  - Fault tolerance: "Failure is the norm, not the exception"
  - Files are huge
  - Most files appended, not overwritten
  - Read over write ratio (Very High!)

GFS Interface

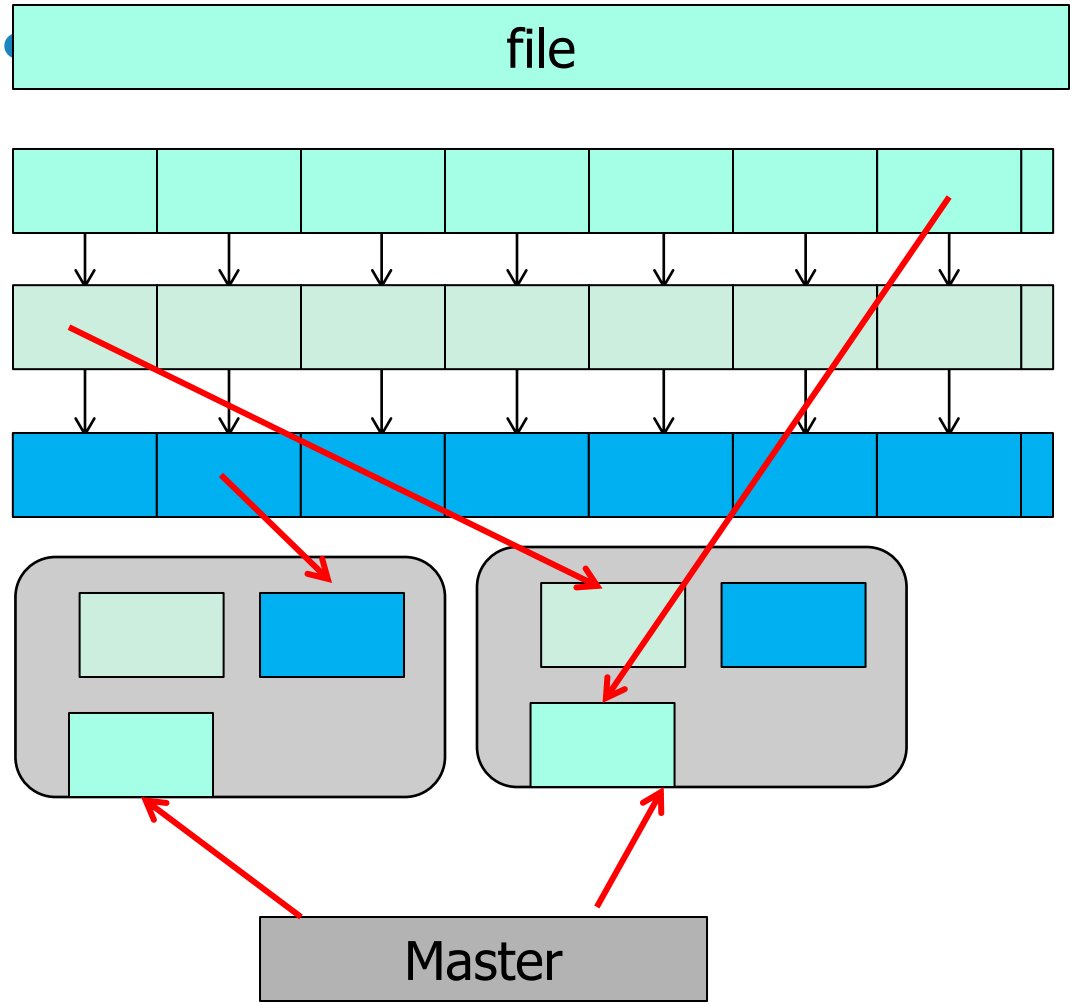
- GFS does not have a standard OS-layer API
  - NO Posix API
  - No kernel/vfs for accessing files
  - User level API to access files
  - GFS servers are implemented in **user space** using native Linux FS
- Files organized in directories
- Operations:
  - Basic: Create, delete, open, close, read, write
  - Additional: Snapshot & append

GFS Master & Chunkservers

- GFS cluster
  - Multiple chunkservers
    - Data storage: fixed-size chunks
    - Chunks replicated on several systems
  - One master
    - Stores file system metadata (names, attributes)
    - Maps files to chunks



GFS Files



A file

Is made of 64Mb/Chunk

That are replicated!!

Chunkservers

Master → takes care of file system namespace
name → {chunk list)

GFS: Chunks and Chunkservers

- Chunk size = 64 MB (default)
  - Chunkserver stores a 32-bit checksum with each chunk
    - In memory & logged to disk: allows it to detect data corruption
- Chunk Handle
  - Globally unique 64-bit number
  - Assigned by the master when the chunk is created
- Chunkservers store chunks on local disks as Linux files
- Each chunk is replicated on multiple chunkservers
  - Three replicas (different levels can be specified)
  - Popular files may need more replicas to avoid hotspots

GFS: Master

- Maintains all file system metadata
  - Namespace
  - Access control info
  - Filename to chunks mappings
  - Current locations of chunks
- Manages
  - Chunk leases (locks)
  - Garbage collection (freeing unused chunks)
  - Chunk migration (copying/moving chunks)
- Master replicates its data for fault tolerance
- Periodically communicates with all chunkservers
  - Via heartbeat messages
  - To get state and send commands

GFS: Client interaction model

- Clients connects directly with chunkservers
- No data caching at server/client
  - System buffer cache will do it anyway
- Clients cache metadata

GFS: Master

- All metadata stored **in memory**
- Namespaces & name-to-chunk maps
- Chunk locations are not persistent
  - Queried from the chunkservers

Reading files:

1. Contact the master
2. Get the list of chunk handles
   - And file´s metadata
3. Get the location of each of the chunk handles
   - Replicated info
4. Contact ANY available chunkserver

Writing to file

- Master grants a chunk lease to one of the replicas:
  - This replica will be the primary replica
  - Master increases the chunk version number and informs replicas

Writing to files: 2 phases

- Control phase
  - send data(but don´t write to file)
- A client is given a list of replicas
- Client writes to the closest replica
  - Replica forwards data to another replica
  - That replica forwards to another chunkserver
- Chunkservers store this data in a cache

Writing to files: 2 phases

- Data phase
  - Write data.
- Client waits for replicas to acknowledge receiving data.
- Send a write request to the primary
- The primary is responsible for serialization of writes
  - Assigns consecutive serial numbers to all writes
    - Used to detect stale values after a partition
- Once all acks have been received, the primary acknowledges the client

Filesystem Use cases

- GPFS
- Dropbox
- Googlefs
- **HDFS**

Hadoop Distributed File System (HDFS)

- Storage system for Hadoop applications
- The Apache Hadoop software library is a framework that allows for the distributed processing of large data sets across clusters of computers
- Hadoop also includes:
  - **YARN**: A framework for job scheduling and cluster resource management
  - **MapReduce**: A YARN-based system for parallel processing of large data sets

http://hadoop.apache.org

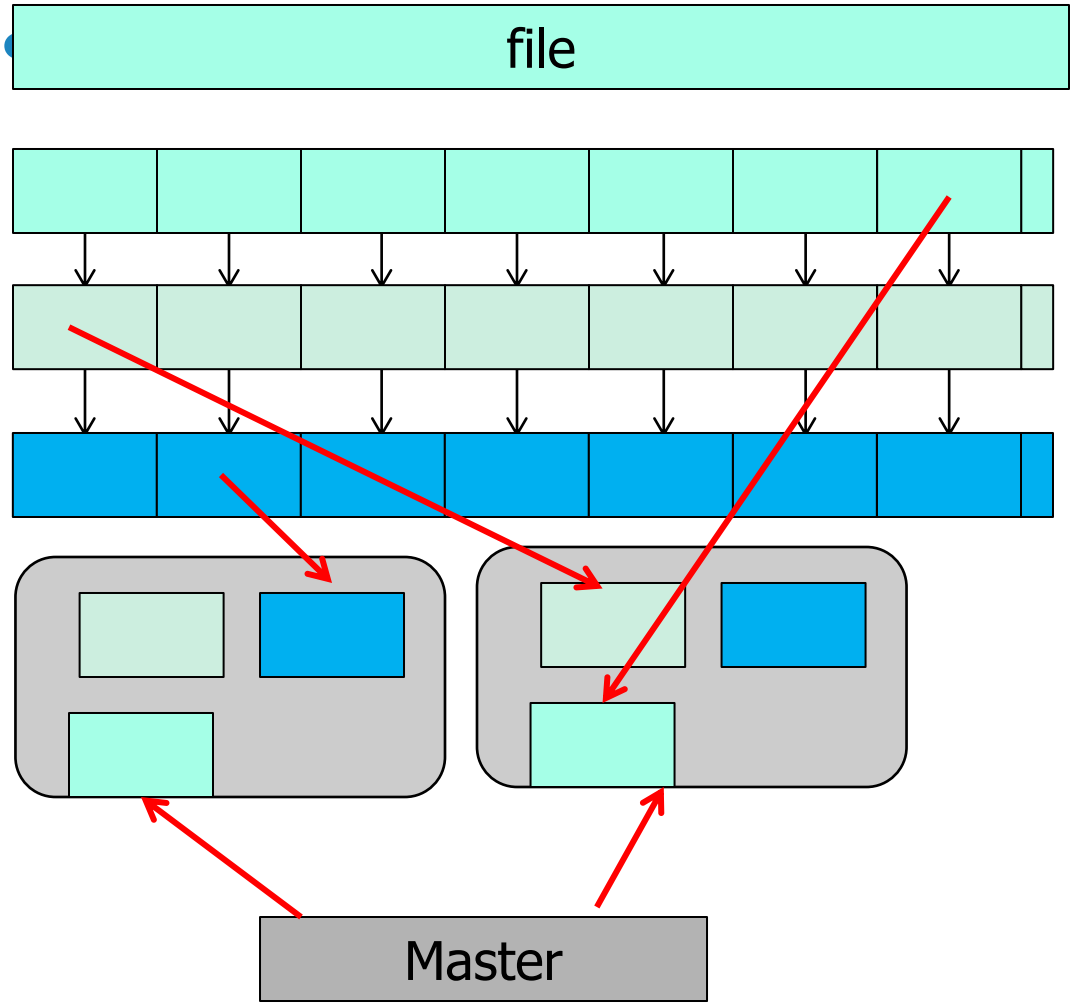
HDFS designed to:

- Run on commodity hardware
- Fault tolerant
- High throughput vs low latency
- No Posix Compliant
- Support large data sets
  - File size of GB – TB & 100s servers
- Write-once-read-many access model for files
- A file´s content need not be changed
  - Except for appends and truncates
  - Suitable for MapReduce apps or web crawlers
  - Simplifies data coherency & enables high throughput
- "Moving computation is cheaper than moving data"

HDFS architecture

- An HDFS cluster consists of:
  - 1 NameNode server
    - Manages the file system namespace
    - Regulates access to files by clients
  - N DataNodes: ~1/node in the cluster
    - Manages storage attached to it
- Offers a file system namespace
  - Where user store data as files
- Files split into 1 or more blocks stored in DataNodes

Do you remember GFS?



A file

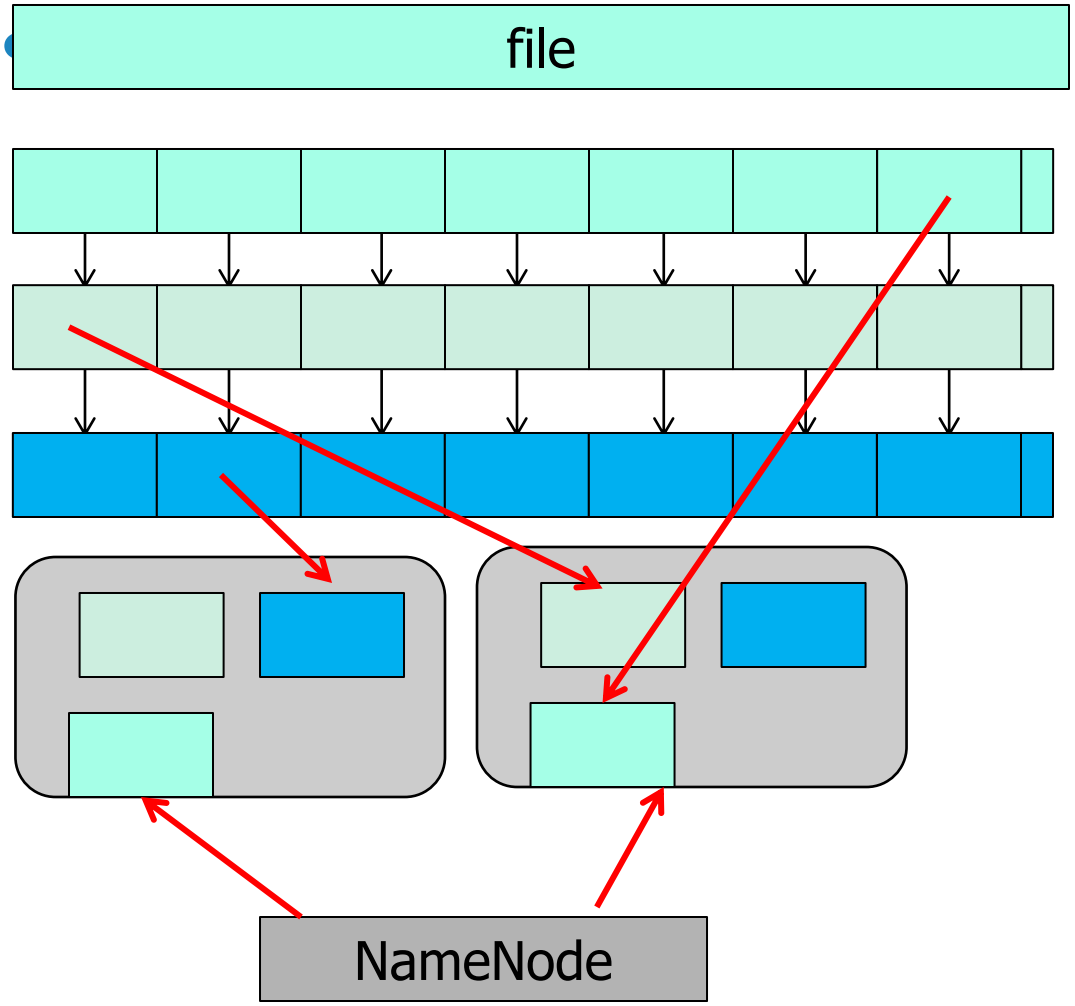
Is made of 64Mb/Chunk

That are replicated!!

Chunkservers

Master → takes care of file system namespace

HDFS files



A file

Is made of 128Mb/Block

That are replicated!!

DataNodes

NameNode→ takes care of file system namespace

NameNode

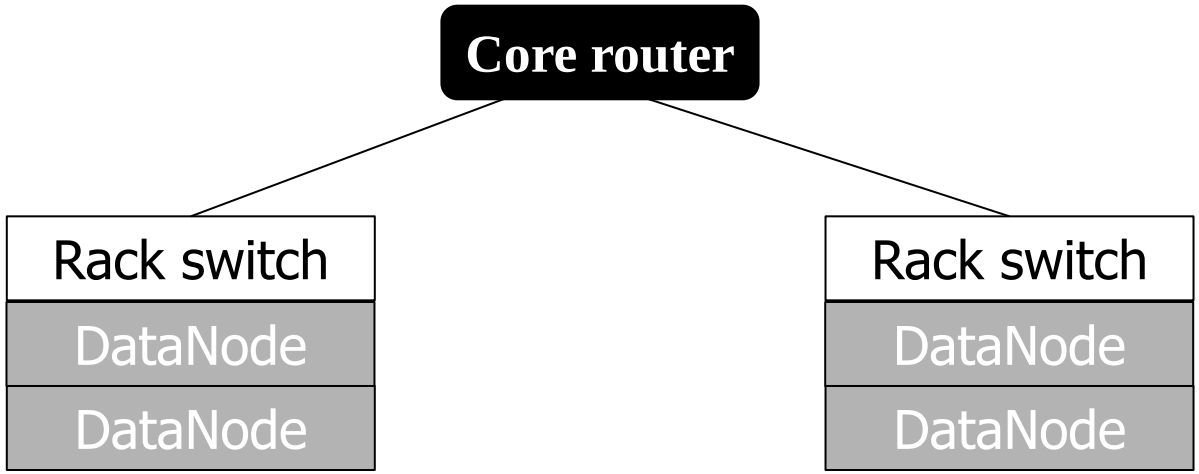
- Manage file system namespace operations:
  - Open, close
  - Renaming files & directories
  - Mapping of blocks to DataNodes
- Heartbeat and Blockreport from DataNodes
  - Detect malfunctions
  - List of all blocks on a DataNode

DataNode

- Manage storage attached to the node
- Responsible for:
  - Serving Read & Write requests from FS´s clients
  - Block creation, deletion & replication
    - When requested from NameNode
- Store blocks´s files in local file system
  - Configurable block size (default 128Mb)
- Blocks are replicated
  - #replicas per file is configurable

Network topology consideration

- Hadoop components are rack-aware



Data replication

- Replica placement (Rack-aware)
  - 3 replicas:
    - 2 in different nodes in local rack
    - 1 in different rack
  - More than 3 replicas:
    - 3 like before
    - Remaining replicas in random places keeping #replicas/rack below a threshold
  - Max replicas == #DataNodes

Data replication

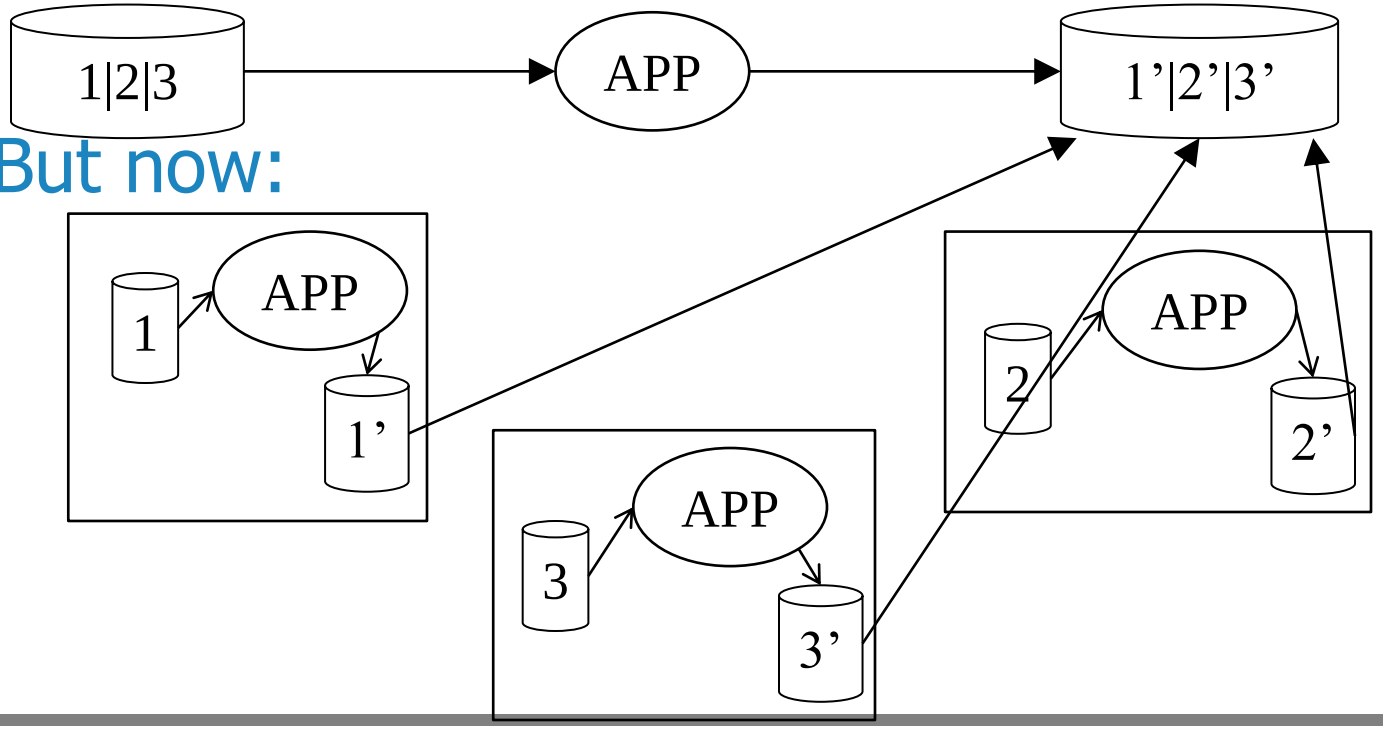
- Replica selection
  - Client sends request to NameNode
    - Receives list of blocks and replica DataNodes per block
  - Client tries to read from the closest replica
    - Same rack
    - Local data center

MapReduce

- Software framework for writing applications to process vast amounts of data in-parallel
- Split input data-set into independent chunks
- Chunks processed by map tasks in parallel
- Map outputs are input to the reduce tasks
- Both input and output stored in file-system
- Framework takes care of scheduling tasks, monitoring them and re-executes the failed tasks
- Typically the compute nodes and the storage nodes are the same

MapReduce

- Software framework for writing applications to process vast amounts of data in-parallel
- Usually:
- But now:



Database Use Case

- Cassandra

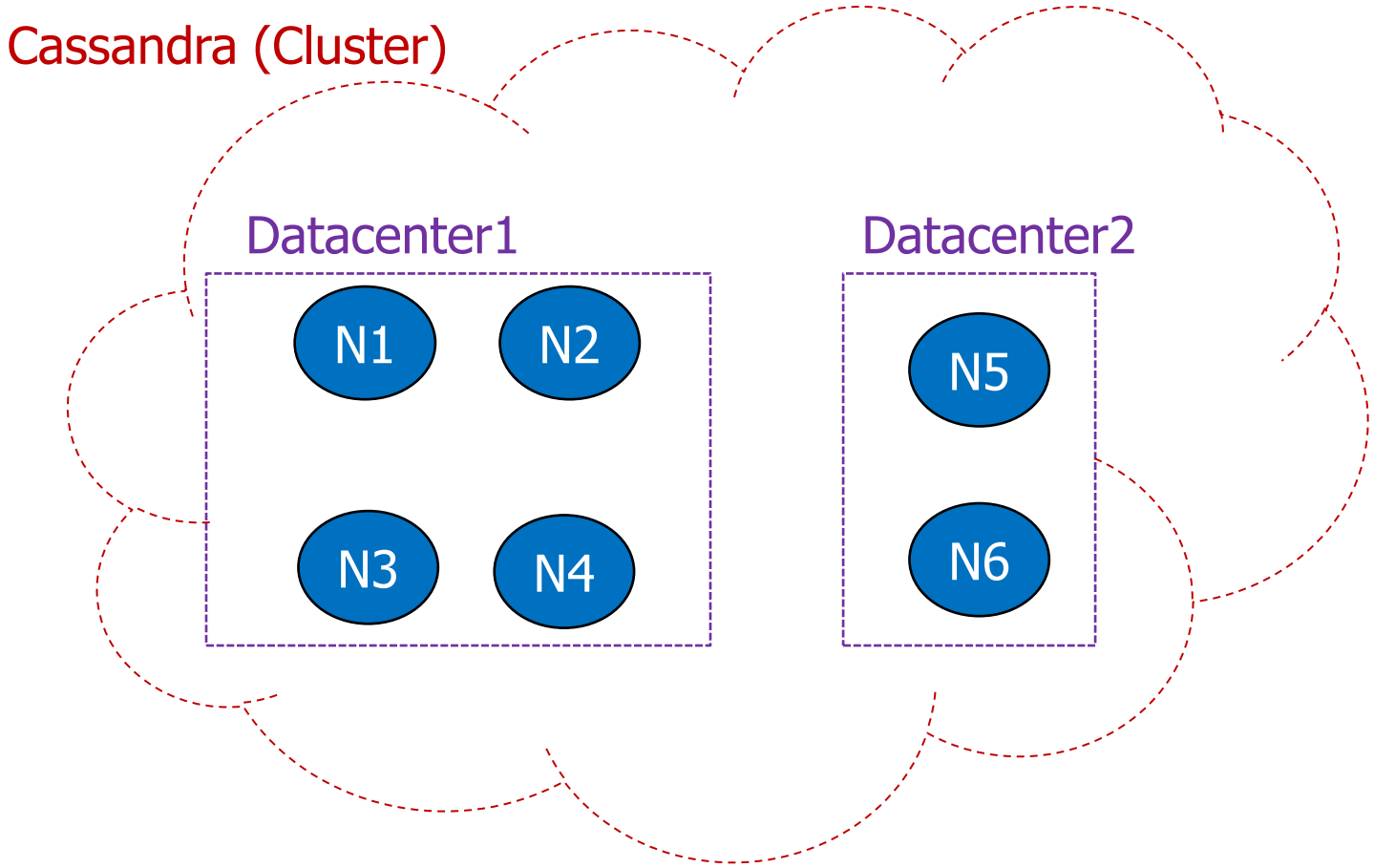
Cassandra

- "A **scalable** multi-master database with no single points of failure"
- A distributed key-value store
- An Apache Hadoop related project
  - https://cassandra.apache.org/
- [Laks10]

Cassandra architecture

- A peer-to-peer distributed system across homogeneous nodes where data is distributed among all nodes in the cluster
- Main elements:
  - Node: where you store your data
  - Datacenter: collection of nodes
    - Granularity used by replication
    - Different workload → use separate datacenters
    - Datacenters must never span physical locations
  - Cluster: 1 or more datacenters
- Cassandra is aware of network topology

Cassandra architecture



Key-Value mapping

- User data (value) must have an identifier key
- How does Cassandra know which server contains a specific key-value?
  - Consistent hashing

The problem of hashing

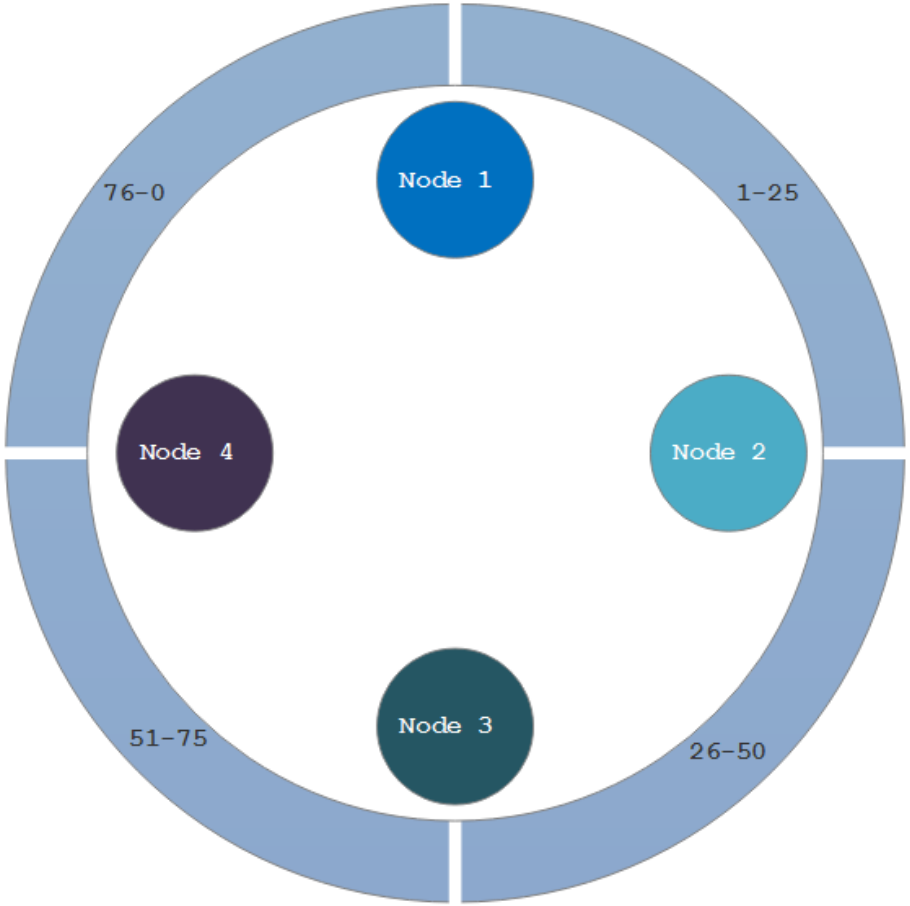
- N servers
- User data with a key
- How do you assign a server to store this data using the key?
  - server(key) = key mod N
    - (for a prime number of servers ;)
- But, what happens when we remove a server? (N=N-1)
  - Data from server must be moved to remaining ones
  - Data from other nodes may need redistribution also!!
    - Because the keyspace has changed!!

Consistent Hashing

- Distribute data across a cluster to minimize reorganization when nodes are added or removed.
- Use a consistent keyspace → a ring
- Data has a **partition key**
- Each partition key → a hash value into ring
  - or *TOKEN*
- Each node in the cluster → range of data based on hash value
  - An interval of keys from the ring
- Cassandra places data on each node according to the partition key and the range that the node is responsible for.

Consistent Hashing Example

http://adamhutson.com/cassandra-partitioning-and-clustering-keys-explained/



How Cassandra works?

- Automatic data distribution
  - Partition Key
  - Clustering Key
  - Ex: `PRIMARY KEY (country_code, state_province, city, gym_name)`
- Partitioned **row** store database
  - any authorized user can connect to any node in any datacenter
  - access data using Cassandra Query Language (CQL)

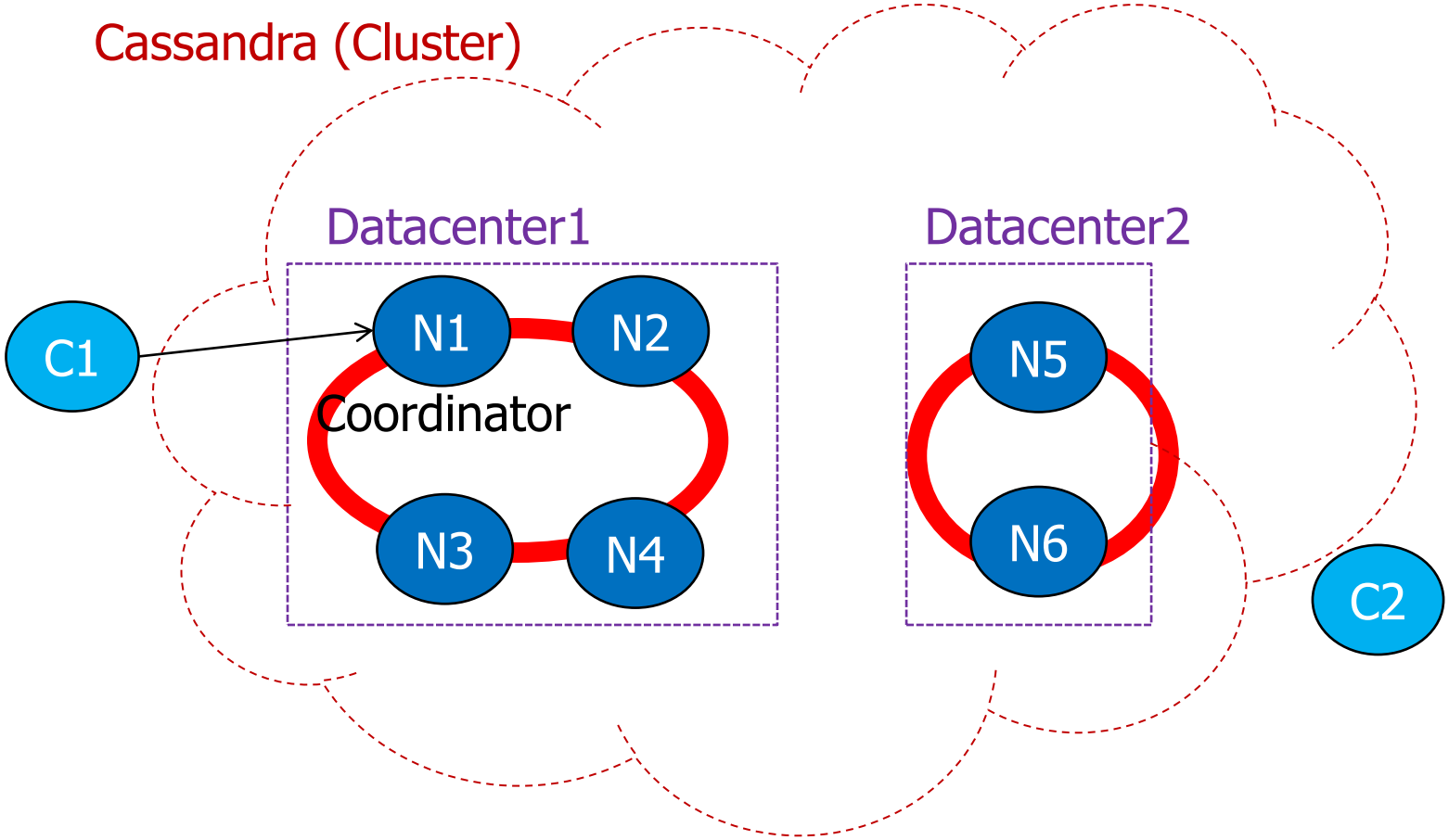
Scalability

- Gossip: peer-to-peer communication protocol to discover and share location and state information about the other nodes in a Cassandra cluster
- Adding nodes (or *bootstrapping*)
  - Configure new node with a *seed*
    - *seed* → any node on the cassandra cluster
  - Tokens are reassigned automatically
- Removing nodes
  - Use nodetool to remove node
  - Tokens ara reassigned automatically

How Cassandra works?

- Client read or write requests can be sent to **any node** in the cluster
- When a client connects to a node with a request, that node serves as **the** coordinator for that particular client operation
- Coordinator acts as a proxy between client application and nodes that own the data being requested
- Coordinator determines which nodes in the ring should get the request based on how the cluster is configured
  - Basically due to the specific consistency used

How Cassandra works? Architecture

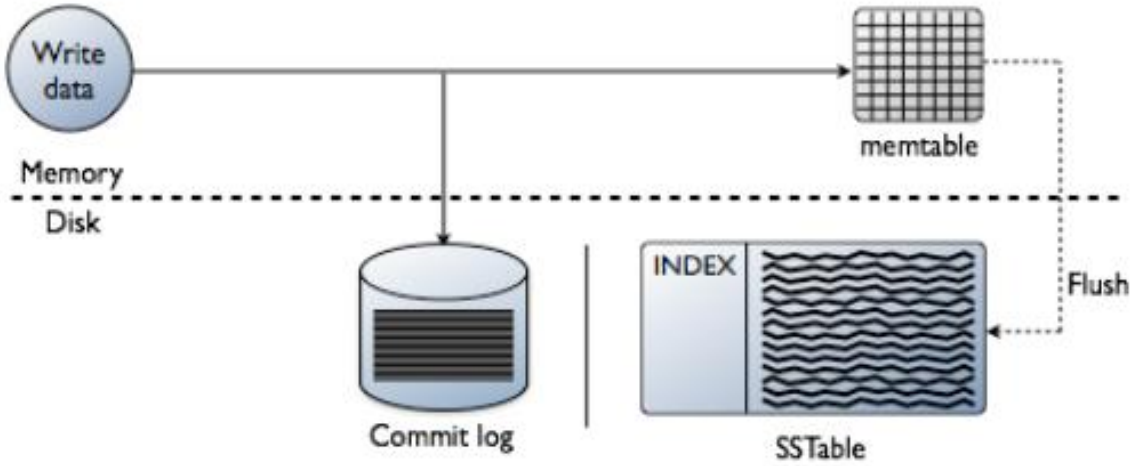


Writing data

- Coordinator sends write request to *all* replicas
- Write consistency level determines how many replica nodes must respond with a success acknowledgment for the write to be considered successful
- Success means that the data was written to the *commit log* and the *memtable*

Cassandra: How is data written?

- Logging data in the *commit log*
- Writing data to *memtable*
- Flushing data from memtable
- Storing data on disks in *SSTables*
  - *Sorted String Table*
  - immutable data file
  - append only and stored on disk sequentially



Cassandra: How is data written?

- A sequentially written *commit log* on each node captures write activity to ensure data durability.
- Data is then indexed and written to an in-memory structure, called a *memtable*
- Each time the memory structure is full, the data is written to disk in an *SSTable* data file
- All writes are automatically partitioned and **replicated** throughout the cluster

Data replication

- *Replication factor*: total number of replicas across the cluster
  - 1 → only one copy of each row in the cluster
  - 2 → 2 copies of each row, each copy on a different node.
- Replica placement
  - Partitioner: determines which node will receive the first replica of a piece of data, and how to distribute other replicas across other nodes in the cluster
    - (The hash function)
  - Replica placement strategy: determines which nodes to place replicas on.
    - *SimpleStrategy*: Single datacenter and 1 rack.
    - *NetworkTopologyStrategy*: Remaining cases.

Cassandra consistency models

- Fully configurable consistency on:
  - *Cluster*
  - *Datacenter*
  - Or, per individual read or write *operation*
- By default: ONE for all read & write ops
- QUORUM = (SumReplicationFactors / 2) + 1

Cassandra WRITE consistency levels

- A write must be written to…
  - ALL: …all replica nodes in the cluster for that partition
  - "QUORUM": …a quorum of replica nodes in…
    - EACH_QUORUM: … EACH datacenter.
    - QUORUM: … ALL datacenter.
    - LOCAL_QUORUM: .. SAME datacenter as coordinator
  - ONE: … at least one replica node
  - TWO: …at least two replica nodes
- LOCAL_ONE: … at least one replica node in the same datacenter
- ANY: … at least one node
  - Note the missing "replica" part!

Cassandra READ consistency levels

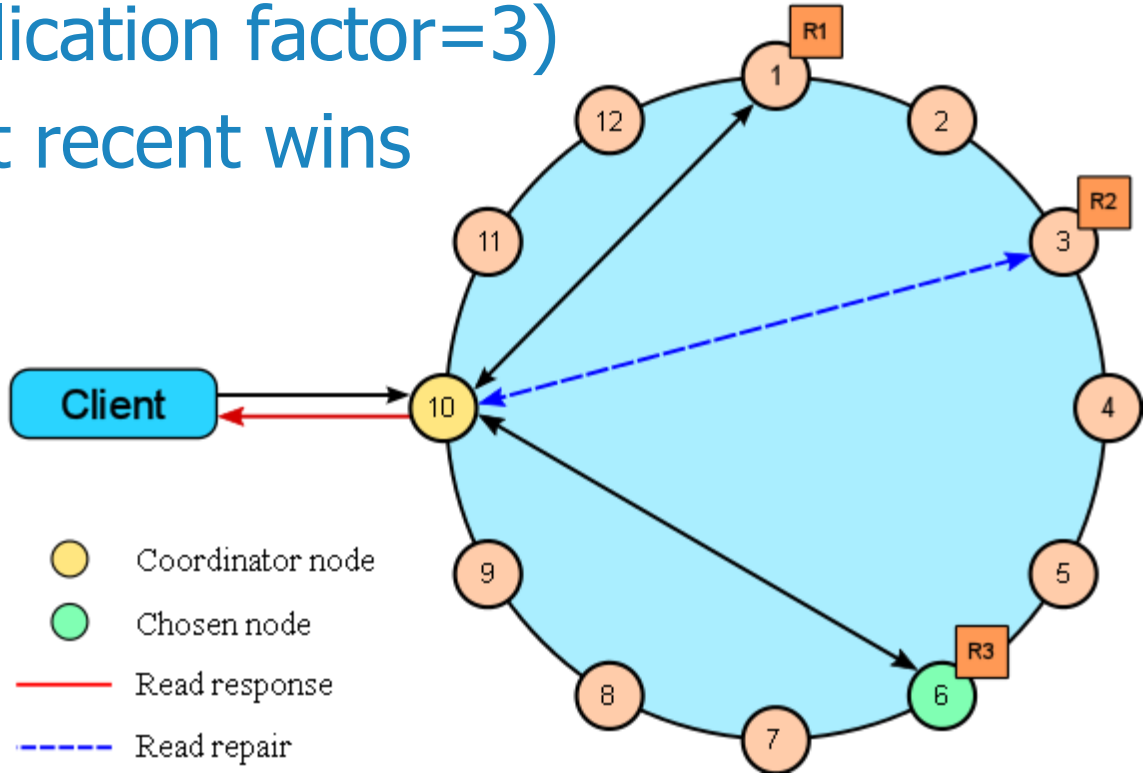
- ALL
- ~~EACH_QUORUM~~
- QUORUM
- LOCAL_QUORUM
- ONE
- TWO
- LOCAL_ONE
- ~~ANY~~

How Cassandra works? Examples

- Reading data
- Writing data

Reading data

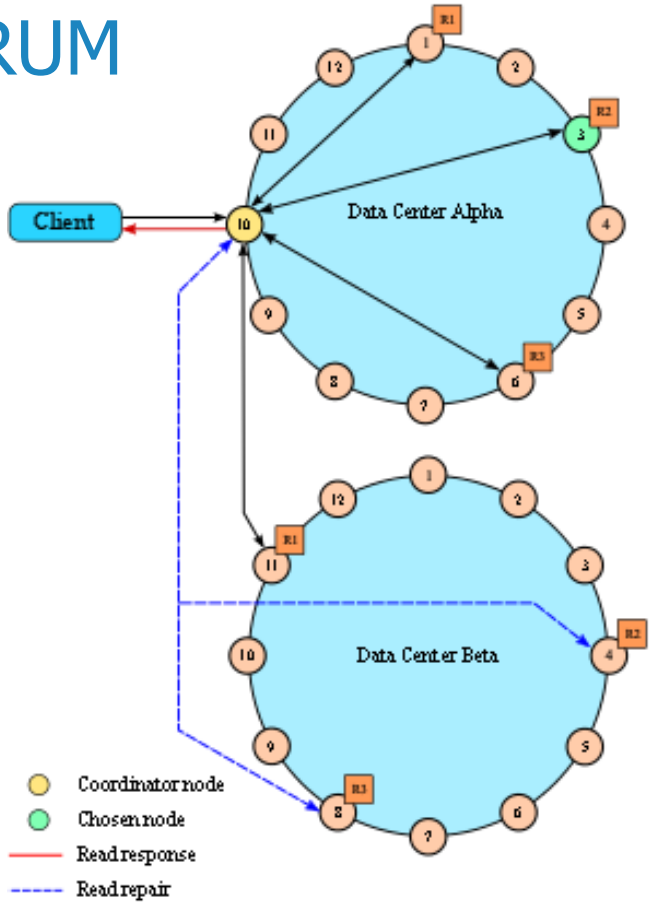
- A single datacenter cluster with a consistency level of QUORUM
- (replication factor=3)
- Most recent wins



https://docs.datastax.com/en/archived/cassandra/2.1/cassandra/dml/dmlClientRequestsRead.html

Reading data

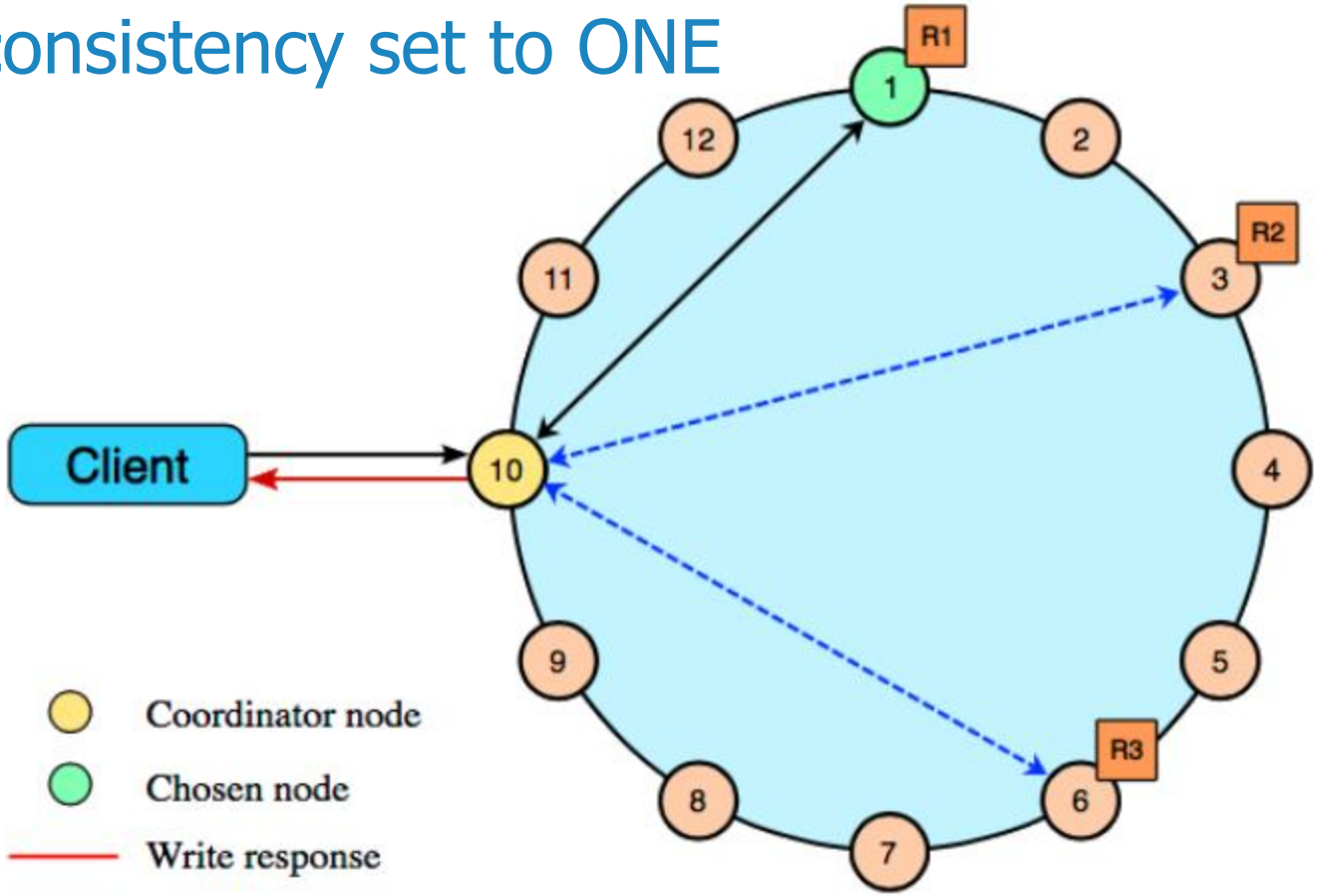
- A two datacenter cluster with a consistency level of QUORUM



https://docs.datastax.com/en/archived/cassandra/2.1/cassandra/dml/dmlClientRequestsRead.html

Writing data

- Single datacenter cluster with 3 replica nodes and consistency set to ONE



https://docs.datastax.com/en/archived/cassandra/2.1/cassandra/dml/architectureClientRequestsWrite.html

Examples Summary

- Filesystem use cases
  - GPFS
  - Dropbox
  - Googlefs
  - HDFS
- Database use case
  - Cassandra

Data management summary

- In-Memory
  - From load/store to In-Memory DB
- On-Disk
  - From in/out to On-Disk DB
- Future?

References

- [RFC1813] *Network File System (NFS) v3 Protocol*. tools.ietf.org/html/rfc1813
- [RFC7530] *Network File System (NFS) v4 Protocol*. tools.ietf.org/html/rfc7530
- [RFC5661] *Network File System (NFS) v4.1 Protocol*. tools.ietf.org/html/rfc5661
- [Tane06] Tannembaum,A.S. *Distributed systems: Principles and Paradigms*. 2006
- [Schm02] Schmuck, F. B., and Roger L. H. *GPFS: A Shared-Disk File System for Large Computing Clusters.* FAST. Vol. 2. No. 19. 2002.
- [Ghem03] Ghemawat, S., Howard G., and Shun-Tak L. *The Google file system*. Vol. 37. No. 5. ACM, 2003.
- [Laks10] Lakshman, A., and Prashant M. *Cassandra: a decentralized structured storage system*. ACM SIGOPS Operating Systems Review 44.2 2010.
- [Remz18] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau.*Operating Systems: Three Easy Pieces.* Arpaci-Dusseau Books. August, 2018 (V. 1.00)

Addendum…

Disk arrays: main ideas

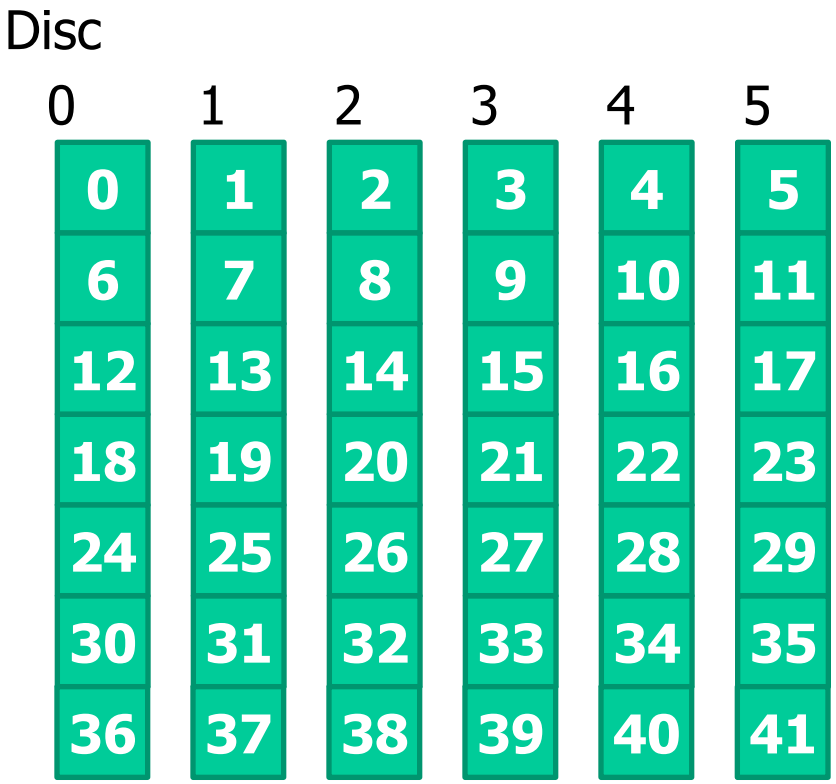
- RAID
  - **R**edundant **A**rray of **I**nexpensive **D**iscs
- Objective
  - Use several discs as a single "device"
  - Increase the bandwidth of the storage system
    - RAID_BW ≈ Disc_BW * Number of discs
- Main problem
  - More discs used ➔ Higher probability for disc fault
    - Several tolerance levels

Disc arrays: potential parallelism

- Large requests
  - Parallelism within a request
  - A part of the request can be done by each disk
    - It will reduce the transfer time
    - I will not reduce the seek time
- Small requests
  - Parallelism among requests
  - Several requests can be done at the same time
    - As long as there is no overlap on the discs used
    - Seeks done in parallel
    - Transferences done in parallel

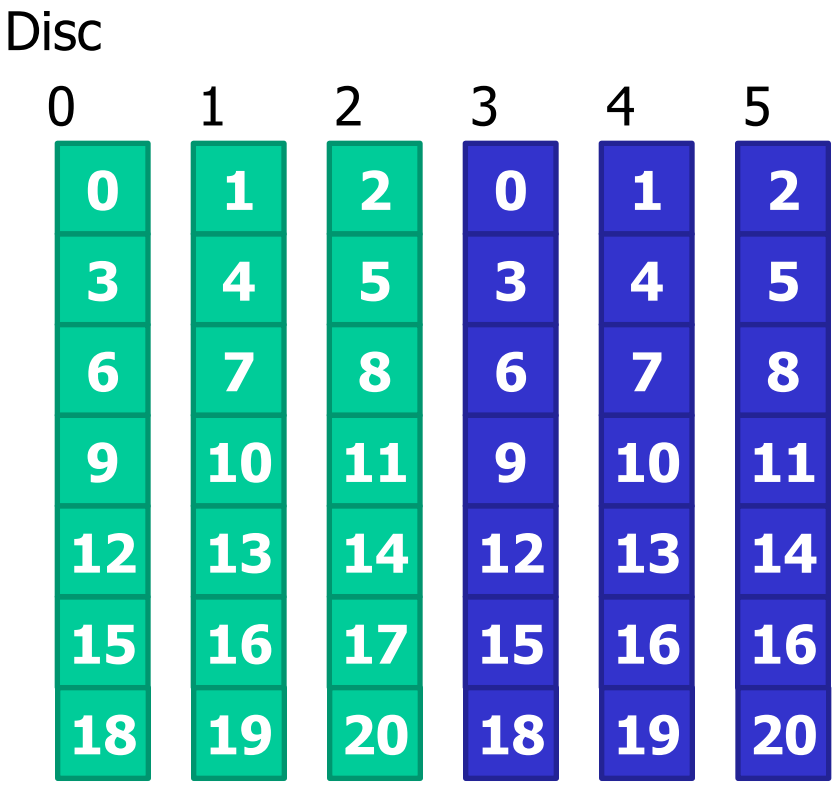
RAID0

- Data distribution
  - Round-robin on all discs
- Advantages
  - Highest bandwidth
  - Highest capacity
- Disadvantage
  - Not tolerant to any failure



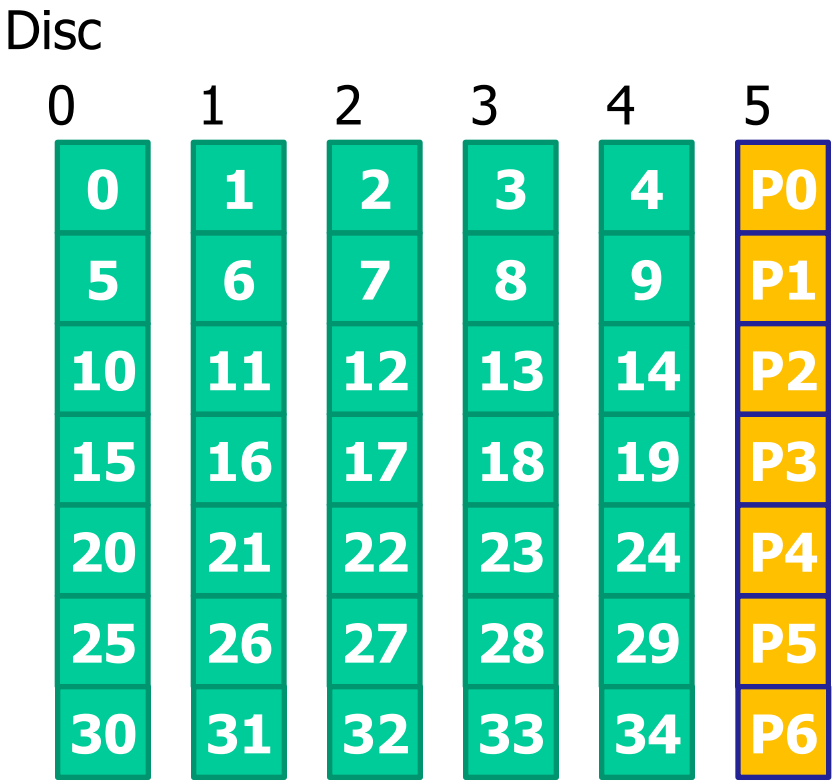
RAID1

- Data distribution
  - Two replicated RAID0
- Writes
  - Done on both copies
- Reads
  - From any of the copies
  - Possible optimizations
- Advantages
  - Fault tolerant
- Disadvantages
  - Less parallelism
  - Wasted space



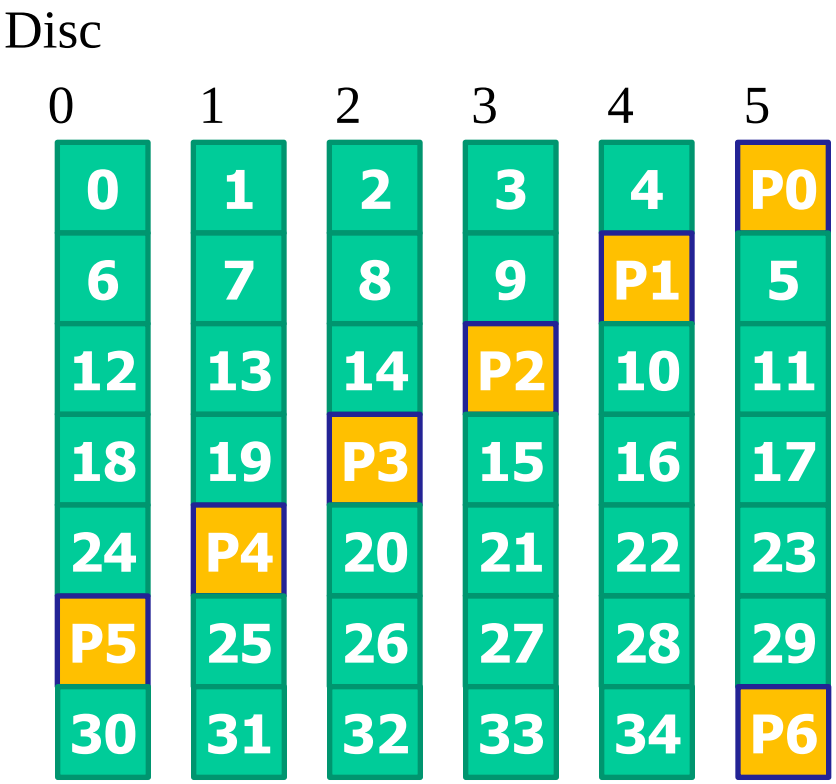
RAID4

- Data distribution
  - RAID0 plus a parity discs
  - Parity computed using **XOR**
- Writes
  - modified data
  - Newly parity block
- Advantages
  - Allows one disc failure
- Disadvantage
  - No write parallelism
    - Always use parity disc



RAID5

- Data distribution
  - RAID4 plus **interleaved parity**
- Advantages
  - Good performance
    - Similar to RAID0
- Disadvantage
  - Only allows one disc failure
  - Slow small writes
    - More later ☺



Small-write problem

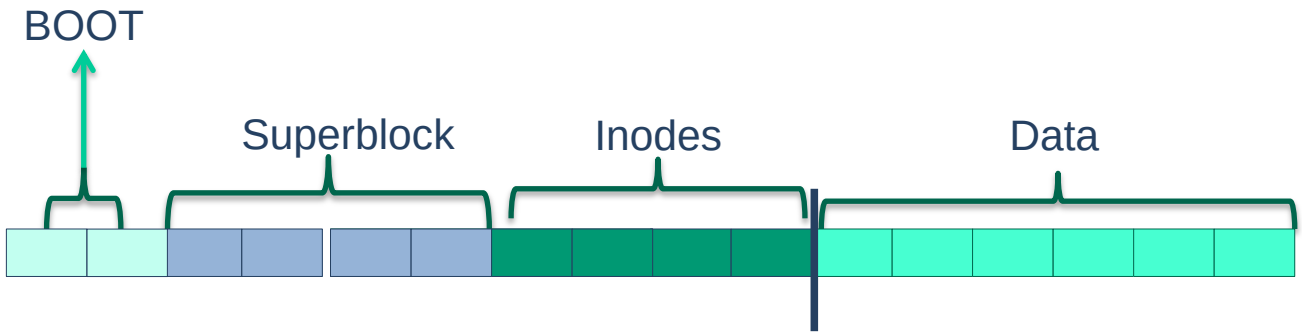
- As we have seen
  - Writing a partial stripe implies
    - Reading data from disc
    - Computing the parity
    - Write the intended data plus the recomputed parity
- Several solutions have been proposed
  - Parity logging
  - Mixing log-structured file systems with disc arrays
  - RAID hierarchies (i.e. AutoRAID)

Log-structured FS & small writes

- Take advantage of LSF to always write full stripes
- Keep a log
  - Write all blocks in a log
    - Special disc or cache
  - Once the log is as long as a stripe ➔ write it
    - Including parity computation and modified data
- Two well know systems use this technique
  - Zebra
  - xFS (Part of the NOW project)

Example: Ext2

- The general organization of the UNIX file system implementation on a logical disk of contiguous disk blocks.



Example: Ext2

- Boot sector
- Superblock
  - FileSystem information: block size, #inodes, #free inodes, #data blocks, #free blocks, …
- Inodes
  - Multilevel Indexed blocks
- Data Blocks
- Directory
  - Maps a filename with its inode
  - All file attributes found in the inode

Example: Ext2

- Inode
  - Block with file information
    - Size, type, prots, …
  - Indexes to data blocks
    - (1 – 4Kb)
    - 10 direct indexes
      - Filesize: 10-40Kb
    - 1 indirect index
      - 256 – 1024 blocks→ 256Kb–4MB
    - 1 double indirect index
      - 65K-1M blocks → 65Mb – 4Gb
    - 1 triple indirect index
      - 16M – 1G blocks → 16Gb – 4Tb

